PARTE PRÁCTICA (27 minutos)

Ejercicio: Crear la estructura raíz del repositorio AceriaApp y la estructura inicial del punto 1.1 en la carpeta modulo-1-introduccion-microservicios/1.1-monolito-vs-microservicios/, con una solución, un proyecto de API monolítico de referencia, un proyecto de tests, la documentación de la decisión arquitectónica y el README del punto.

Ubicación de trabajo: C:\cursos\

Paso 1 — Crear la carpeta raíz AceriaApp/

Acción 1: Abrir la terminal integrada de Visual Studio 2026 con Ctrl+ñ (o Ctrl+`` en teclados no españoles).
Acción 2: Escribir cd C:\cursos y pulsar Enter para situarse en la carpeta de trabajo.
Acción 3: Escribir mkdir AceriaApp y pulsar Enter.
Acción 4: Escribir cd AceriaApp y pulsar Enter.
Acción 5: Escribir dir y pulsar Enter para verificar que la carpeta está vacía y es la ubicación actual.

Por qué se hace: el prompt maestro define AceriaApp/ como la raíz del repositorio del curso. Toda la estructura de módulos y puntos cuelga de esta carpeta raíz. Crearla primero garantiza que todos los comandos posteriores se ejecuten en la ubicación correcta.

Errores comunes:

· Crear la carpeta en una ruta distinta a C:\cursos: los comandos posteriores fallan porque las rutas relativas no coinciden.
· No cambiar a la carpeta recién creada con cd AceriaApp: los siguientes mkdir se ejecutan en C:\cursos y ensucian el directorio padre.
· Escribir AceríaApp con tilde: la ruta canónica del curso es AceriaApp sin tilde.

Verificación: al ejecutar dir, la salida no debe mostrar ningún archivo ni carpeta. La ruta actual debe ser C:\cursos\AceriaApp.

Paso 2 — Crear el README.md raíz del repositorio

Acción 1: En la carpeta raíz AceriaApp, hacer clic derecho y seleccionar New → Text Document.
Acción 2: Renombrar el archivo a README.md, incluyendo la extensión .md.
Acción 3: Abrir el archivo con Visual Studio 2026.
Acción 4: Escribir exactamente el siguiente contenido:

AceriaApp — Curso de Microservicios con .NET 8

Repositorio del curso formativo de 30 horas sobre microservicios con .NET 8, organizado en 8 módulos y 34 puntos.

Estructura del repositorio

- `modulo-1-introduccion-microservicios/`: 5 puntos (1.1 a 1.5).
- `modulo-2-apis-rest-aspnet-core/`: 4 puntos (2.1 a 2.4).
- `modulo-3-arquitectura-hexagonal/`: 4 puntos (3.1 a 3.4).
- `modulo-4-patrones-arquitectura/`: 5 puntos (4.1 a 4.5).
- `modulo-5-ddd-cqrs/`: 5 puntos (5.1 a 5.5).
- `modulo-6-comunicacion-microservicios/`: 3 puntos (6.1 a 6.3).
- `modulo-7-observabilidad-resiliencia/`: 4 puntos (7.1 a 7.4).
- `modulo-8-api-gateway/`: 4 puntos (8.1 a 8.4).

Cada punto contiene:

- `README.md`: objetivo del punto, estado del proyecto, cómo ejecutarlo y qué se ha añadido respecto al punto anterior.
- `AceriaApp.sln`: solución completa con todos los proyectos generados hasta ese punto.
- `src/`: código fuente de todos los proyectos.
- `tests/`: tests generados hasta ese punto.
- `docs/`: ADRs, diagramas UML y documentación generada.

Requisitos

- .NET 8 SDK (LTS).
- Visual Studio 2026 Community.
- SQL Server Express (incluido en Visual Studio 2026 Community).
- Docker Desktop (para tests de integración con Testcontainers).

Cadena de conexión

Server=.\SQLEXPRESS;Database=AceriaDb;Trusted_Connection=True;TrustServerCertificate=True

Cómo ejecutar cualquier punto

1. Abrir la carpeta del punto en Visual Studio 2026.
2. Abrir `AceriaApp.sln`.
3. Verificar que SQL Server Express está arrancado.
4. Ejecutar `dotnet ef database update` en el proyecto de API correspondiente.
5. Ejecutar `dotnet run` en el proyecto de API.
6. Navegar a la URL de Swagger que indique la salida de `dotnet run`.

Acción 5: Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: el prompt maestro define un README.md en la raíz del repositorio que explica el curso y la estructura. Es el punto de entrada del alumno y debe existir antes de crear cualquier módulo o punto.

Errores comunes:

· Guardar el archivo como README.md.txt: Windows añade la extensión .txt por defecto. Hay que forzar la extensión .md.
· Escribir el contenido sin los bloques de código para la cadena de conexión: el Markdown no se renderiza correctamente.
· Omitir la sección de cadena de conexión: el alumno no sabe qué cadena usar.

Verificación: el archivo README.md debe existir en C:\cursos\AceriaApp\ y al abrirlo debe mostrar el título, la estructura, los requisitos, la cadena de conexión y las instrucciones de ejecución.

Paso 3 — Crear la estructura de carpetas del módulo y del punto

Acción 1: En la terminal, verificar que la ruta actual es C:\cursos\AceriaApp con cd y pulsar Enter.
Acción 2: Escribir mkdir modulo-1-introduccion-microservicios y pulsar Enter.
Acción 3: Escribir mkdir modulo-1-introduccion-microservicios\1.1-monolito-vs-microservicios y pulsar Enter.
Acción 4: Escribir dir modulo-1-introduccion-microservicios y pulsar Enter para verificar que existe la carpeta 1.1-monolito-vs-microservicios.
Acción 5: Escribir cd modulo-1-introduccion-microservicios\1.1-monolito-vs-microservicios y pulsar Enter.

Por qué se hace: cada módulo tiene su propia carpeta en la raíz del repositorio y cada punto tiene su propia carpeta dentro del módulo. Esta separación permite que cada punto sea autocontenido y ejecutable sin depender de otros.

Errores comunes:

· Escribir la ruta con barras inclinadas (/) en lugar de barras invertidas () en Windows: PowerShell acepta ambas, pero cmd solo acepta .
· Crear la carpeta con un nombre distinto al slug indicado: el slug debe ser exactamente 1.1-monolito-vs-microservicios.
· Olvidar cambiar a la carpeta del punto con cd: los siguientes comandos se ejecutan en la carpeta del módulo.

Verificación: al ejecutar dir modulo-1-introduccion-microservicios, debe aparecer la carpeta 1.1-monolito-vs-microservicios. La ruta actual debe ser C:\cursos\AceriaApp\modulo-1-introduccion-microservicios\1.1-monolito-vs-microservicios.

Paso 4 — Crear la solución AceriaApp.sln dentro del punto

Acción 1: Escribir dotnet new sln -n AceriaApp y pulsar Enter.
Acción 2: Escribir dir y pulsar Enter para verificar que se ha creado AceriaApp.sln.
Acción 3: Verificar que la salida de dotnet new sln muestra The template "Solution File" was created successfully.

Por qué se hace: la solución es el contenedor que agrupa todos los proyectos del punto. El nombre AceriaApp se mantiene constante en todos los puntos para que el alumno reconozca la solución al abrir cualquier carpeta.

Errores comunes:

· Ejecutar dotnet new sln sin -n AceriaApp: genera una solución con el nombre de la carpeta (1.1-monolito-vs-microservicios.sln), lo que rompe la convención.
· Ejecutar el comando desde la raíz del repositorio en lugar de dentro de la carpeta del punto: la solución se crea en el lugar equivocado.
· Usar dotnet new solution en lugar de dotnet new sln: el alias sln es el correcto.

Verificación: el archivo AceriaApp.sln debe existir en la carpeta actual.

Paso 5 — Crear el proyecto de API monolítico de referencia

Acción 1: Escribir mkdir src y pulsar Enter.
Acción 2: Escribir cd src y pulsar Enter.
Acción 3: Escribir dotnet new web -n AceriaApp.Monolito.Api y pulsar Enter.
Acción 4: Escribir cd .. y pulsar Enter para volver a la carpeta del punto.
Acción 5: Escribir dir src y pulsar Enter para verificar que existe la carpeta AceriaApp.Monolito.Api.

Por qué se hace: el proyecto monolítico de referencia sirve como punto de partida del curso y como término de comparación con los microservicios que se construirán en módulos posteriores. El nombre AceriaApp.Monolito.Api deja claro que se trata del monolito.

Errores comunes:

· Usar dotnet new webapi en lugar de dotnet new web: webapi genera controladores, mientras que el curso usa Minimal API.
· Crear el proyecto fuera de src/: rompe la estructura del repositorio definida en el prompt maestro.
· Usar dotnet new console: genera una aplicación de consola, no una API web.

Verificación: debe existir la carpeta src\AceriaApp.Monolito.Api con los archivos Program.cs, AceriaApp.Monolito.Api.csproj y appsettings.json.

Paso 6 — Añadir el proyecto monolítico a la solución

Acción 1: Escribir dotnet sln AceriaApp.sln add src\AceriaApp.Monolito.Api\AceriaApp.Monolito.Api.csproj y pulsar Enter.
Acción 2: Escribir dotnet sln AceriaApp.sln list y pulsar Enter.
Acción 3: Verificar que la salida muestra src\AceriaApp.Monolito.Api\AceriaApp.Monolito.Api.csproj.

Por qué se hace: añadir el proyecto a la solución permite compilarlo y ejecutarlo desde Visual Studio 2026 y desde la línea de comandos con dotnet build AceriaApp.sln.

Errores comunes:

· Escribir la ruta sin la extensión .csproj: dotnet sln add requiere la ruta completa al archivo de proyecto.
· Ejecutar el comando desde src/ en lugar de la raíz del punto: la solución no se encuentra.
· Olvidar el nombre de la solución (AceriaApp.sln): el comando falla si hay más de una solución en la carpeta.

Verificación: la salida de dotnet sln list debe mostrar el proyecto AceriaApp.Monolito.Api con su ruta relativa.

Paso 7 — Crear el proyecto de tests

Acción 1: Escribir mkdir tests y pulsar Enter.
Acción 2: Escribir cd tests y pulsar Enter.
Acción 3: Escribir dotnet new xunit -n AceriaApp.Monolito.Tests y pulsar Enter.
Acción 4: Escribir cd .. y pulsar Enter para volver a la carpeta del punto.
Acción 5: Escribir dir tests y pulsar Enter para verificar que existe la carpeta AceriaApp.Monolito.Tests.

Por qué se hace: el proyecto de tests albergará las pruebas unitarias y de integración del monolito. Se crea desde el primer punto para que la práctica de TDD forme parte del flujo de trabajo desde el inicio.

Errores comunes:

· Usar dotnet new mstest o dotnet new nunit: el curso usa xUnit como framework de pruebas.
· Crear el proyecto en src/ en lugar de tests/: rompe la separación entre código de producción y código de pruebas.
· Usar dotnet new xunit sin -n: genera un proyecto con el nombre de la carpeta (tests), lo que rompe la convención.

Verificación: debe existir la carpeta tests\AceriaApp.Monolito.Tests con el archivo AceriaApp.Monolito.Tests.csproj y el archivo UnitTest1.cs.

Paso 8 — Añadir el proyecto de tests a la solución y referenciar el monolito

Acción 1: Escribir dotnet sln AceriaApp.sln add tests\AceriaApp.Monolito.Tests\AceriaApp.Monolito.Tests.csproj y pulsar Enter.

Acción 2: Escribir dotnet add tests\AceriaApp.Monolito.Tests\AceriaApp.Monolito.Tests.csproj reference src\AceriaApp.Monolito.Api\AceriaApp.Monolito.Api.csproj y pulsar Enter.
Acción 3: Escribir dotnet sln AceriaApp.sln list y pulsar Enter.
Acción 4: Verificar que la salida muestra los dos proyectos.

Por qué se hace: la referencia permite que el proyecto de tests acceda a las clases públicas del proyecto monolítico. Sin esta referencia, los tests no pueden instanciar los tipos que deben verificar.

Errores comunes:

· Añadir la referencia en dirección inversa (del monolito a los tests): crea una dependencia circular.
· Olvidar añadir el proyecto de tests a la solución: dotnet test AceriaApp.sln no encontrará el proyecto.
· Escribir la ruta de referencia sin la extensión .csproj: el comando falla.

Verificación: dotnet sln list debe mostrar AceriaApp.Monolito.Api y AceriaApp.Monolito.Tests.

Paso 9 — Añadir los paquetes NuGet de EF Core al proyecto monolítico

Acción 1: Escribir dotnet add src\AceriaApp.Monolito.Api\AceriaApp.Monolito.Api.csproj package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.0 y pulsar Enter.
Acción 2: Escribir dotnet add src\AceriaApp.Monolito.Api\AceriaApp.Monolito.Api.csproj package Microsoft.EntityFrameworkCore.Design --version 8.0.0 y pulsar Enter.
Acción 3: Escribir dotnet add src\AceriaApp.Monolito.Api\AceriaApp.Monolito.Api.csproj package Swashbuckle.AspNetCore y pulsar Enter.
Acción 4: Escribir dotnet list src\AceriaApp.Monolito.Api\AceriaApp.Monolito.Api.csproj package y pulsar Enter.
Acción 5: Verificar que aparecen los tres paquetes con sus versiones.

Por qué se hace: EF Core es el ORM del curso y SQL Server Express es la base de datos. Design es necesario para las herramientas de migraciones. Swashbuckle genera la documentación OpenAPI.

Errores comunes:

· Instalar una versión preliminar de EF Core (8.0.0-rc.1): el curso usa la versión estable 8.0.0.
· Olvidar el paquete Design: dotnet ef migrations add falla con el error Unable to create an object of type 'AceriaDbContext'.
· Instalar Swashbuckle.AspNetCore sin versión: el comando instala la última estable, que es válida para .NET 8.

Verificación: la salida de dotnet list package debe mostrar Microsoft.EntityFrameworkCore.SqlServer 8.0.0, Microsoft.EntityFrameworkCore.Design 8.0.0 y Swashbuckle.AspNetCore con su versión.

Paso 10 — Configurar la cadena de conexión en appsettings.json

Acción 1: En el panel Solution Explorer de Visual Studio 2026, expandir src → AceriaApp.Monolito.Api.
Acción 2: Hacer doble clic en el archivo appsettings.json para abrirlo.
Acción 3: Escribir exactamente el siguiente contenido:

```
{
  "ConnectionStrings": {
    "AceriaDb": "Server=.\
\SQLEXPRESS;Database=AceriaDb;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Logging": {
    "LogLevel": {
```

```
    "Default": "Information",
    "Microsoft.AspNetCore": "Warning"
    }
  },
 "AllowedHosts": "*"
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que el archivo no muestra el indicador de cambios sin guardar en la pestaña.

Por qué se hace: la cadena de conexión define el servidor, la base de datos y el modo de autenticación. Trusted_Connection=True usa la identidad de Windows, y TrustServerCertificate=True evita errores de certificado en entornos locales.

Errores comunes:

· Escribir Server=.\SQLEXPRESS con una sola barra invertida: en JSON, \ representa una barra literal; una sola barra produce un error de escape.
· Usar Database=Aceria en lugar de Database=AceriaDb: el curso usa AceriaDb como nombre canónico.
· Olvidar TrustServerCertificate=True: SQL Server Express puede rechazar la conexión por certificado no confiable.

Verificación: al abrir appsettings.json, debe verse la sección ConnectionStrings con la clave AceriaDb y la cadena completa.

Paso 11 — Crear la entidad Orden en el proyecto monolítico

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.Monolito.Api.
Acción 2: Seleccionar Add → New Folder y nombrar la carpeta Dominio.
Acción 3: Hacer clic derecho sobre la carpeta Dominio, seleccionar Add → Class y nombrar la clase Orden.cs.
Acción 4: Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.Dominio;

public class Orden
{
    public Guid Id { get; set; }
    public string Cliente { get; set; } = string.Empty;
    public string Material { get; set; } = string.Empty;
    public int Cantidad { get; set; }
    public DateTime FechaCreacion { get; set; }
}
```

Acción 5: Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: la entidad Orden representa la orden de producción. Es la unidad de negocio sobre la que operará el monolito de referencia.

Errores comunes:

· Olvidar el namespace: el archivo no compila porque la clase no se resuelve desde otros archivos.
· Usar int para el Id: el curso usa Guid para evitar colisiones en entornos distribuidos.
· No inicializar Cliente y Material con string.Empty: el compilador emite advertencias de nulabilidad.

Verificación: la clase Orden debe aparecer sin subrayados rojos en el editor y el proyecto debe compilar.

Paso 12 — Crear el DbContext del monolito

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.Monolito.Api.
Acción 2: Seleccionar Add → New Folder y nombrar la carpeta Infraestructura.
Acción 3: Hacer clic derecho sobre la carpeta Infraestructura, seleccionar Add → Class y nombrar la clase AceriaDbContext.cs.
Acción 4: Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.Dominio;
using Microsoft.EntityFrameworkCore;

namespace AceriaApp.Monolito.Api.Infraestructura;

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options)
     : base(options)
    {
    }

    public DbSet<Orden> Ordenes => Set<Orden>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
     modelBuilder.Entity<Orden>().HasKey(o => o.Id);
     modelBuilder.Entity<Orden>().Property(o => o.Cliente).HasMaxLength(100).IsRequired();
     modelBuilder.Entity<Orden>().Property(o => o.Material).HasMaxLength(100).IsRequired();
    }
}
```

Acción 5: Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: el DbContext es el punto de acceso a la base de datos. Define las tablas y sus restricciones, y expone los DbSet que usan los repositorios y los endpoints.

Errores comunes:

· Olvidar el constructor que recibe DbContextOptions<AceriaDbContext>: EF Core no puede instanciar el contexto.
· No configurar las longitudes máximas: SQL Server genera columnas nvarchar(max) que son ineficientes.
· Usar DbSet<Orden> Ordenes { get; set; } sin => Set<Orden>(): el curso usa la forma con expresión para evitar setters.

Verificación: la clase AceriaDbContext debe compilar sin errores y el proyecto debe reconocer Microsoft.EntityFrameworkCore.

Paso 13 — Registrar el DbContext y los endpoints en Program.cs

Acción 1: En el panel Solution Explorer, hacer doble clic en Program.cs.
Acción 2: Escribir exactamente el siguiente contenido:

```
using AceriaApp.Monolito.Api.Dominio;
using AceriaApp.Monolito.Api.Infraestructura;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<AceriaDbContext>(options =>
   options.UseSqlServer(builder.Configuration.GetConnectionString("AceriaDb")));
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
```

```
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.MapGet("/api/v1/ordenes", async (AceriaDbContext db) =>
    await db.Ordenes.AsNoTracking().ToListAsync());

app.MapPost("/api/v1/ordenes", async (AceriaDbContext db, Orden orden) =>
{
   orden.Id = Guid.NewGuid();
   orden.FechaCreacion = DateTime.UtcNow;
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    return Results.Created($"/api/v1/ordenes/{orden.Id}", orden);
});

app.Run();
```

Acción 3: Pulsar Ctrl+S para guardar el archivo.
Acción 4: Verificar que no hay subrayados rojos en las líneas añadidas.

Por qué se hace: registrar el DbContext en el contenedor de dependencias permite que los endpoints lo reciban por inyección. AddEndpointsApiExplorer y AddSwaggerGen habilitan OpenAPI. Los endpoints GET y POST permiten consultar y crear órdenes.

Errores comunes:

· Escribir GetConnectionString("Aceria") en lugar de GetConnectionString("AceriaDb"): devuelve null y EF Core lanza InvalidOperationException.
· Olvidar el using Microsoft.EntityFrameworkCore: UseSqlServer no se resuelve.
· Olvidar app.Run(); al final: el proceso termina inmediatamente y no escucha peticiones.

Verificación: Program.cs debe compilar sin errores y mostrar las llamadas a AddDbContext, AddEndpointsApiExplorer, AddSwaggerGen, los dos MapGet/MapPost y la llamada a app.Run().

Paso 14 — Crear la migración inicial

Acción 1: En la terminal, escribir cd src\AceriaApp.Monolito.Api y pulsar Enter.
Acción 2: Escribir dotnet ef migrations add Inicial --output-dir Infraestructura\Migraciones y pulsar Enter.
Acción 3: Verificar que la salida muestra Done. To undo this action, use 'ef migrations remove'.
Acción 4: Escribir dir Infraestructura\Migraciones y pulsar Enter para ver los archivos generados.

Por qué se hace: la migración inicial crea el esquema de la base de datos a partir del modelo de EF Core. Se guarda en Infraestructura\Migraciones para mantener el orden del proyecto.

Errores comunes:

· Ejecutar dotnet ef migrations add desde la raíz de la solución: falla con No project was found.
· Olvidar --output-dir: las migraciones se crean en la raíz del proyecto y ensucian la estructura.
· No tener Microsoft.EntityFrameworkCore.Design instalado: el comando dotnet ef falla con Unable to create an object of type 'AceriaDbContext'.

Verificación: deben existir los archivos 20250101000000_Inicial.cs y AceriaDbContextModelSnapshot.cs dentro de Infraestructura\Migraciones.

Paso 15 — Aplicar la migración a SQL Server Express

Acción 1: Escribir dotnet ef database update y pulsar Enter.
Acción 2: Verificar que la salida muestra Applying migration '20250101000000_Inicial' y Done..
Acción 3: Abrir SQL Server Management Studio o el explorador de objetos de Visual Studio 2026.
Acción 4: Expandir .\SQLEXPRESS → Databases → AceriaDb → Tables.
Acción 5: Verificar que existe la tabla dbo.Ordenes.

Por qué se hace: dotnet ef database update aplica la migración y crea la base de datos AceriaDb con la tabla Ordenes.

Errores comunes:

· No tener SQL Server Express arrancado: falla con A network-related or instance-specific error.
· Usar una cadena de conexión con Database=Aceria en lugar de AceriaDb: crea una base de datos distinta.
· No tener permisos de CREATE DATABASE: falla con CREATE DATABASE permission denied.

Verificación: la tabla dbo.Ordenes debe existir en la base de datos AceriaDb con las columnas Id, Cliente, Material, Cantidad y FechaCreacion.

Paso 16 — Ejecutar la API y verificar los endpoints

Acción 1: Escribir dotnet run y pulsar Enter.
Acción 2: Verificar que la salida muestra Now listening on: https://localhost:7xxx y Now listening on: http://localhost:5xxx.
Acción 3: Abrir un navegador y navegar a https://localhost:7xxx/swagger.
Acción 4: Verificar que Swagger UI muestra los endpoints GET /api/v1/ordenes y POST /api/v1/ordenes.
Acción 5: Pulsar Ctrl+C en la terminal para detener la API.

Por qué se hace: ejecutar la API verifica que el proyecto compila, que la configuración es correcta y que los endpoints están expuestos.

Errores comunes:

· No confiar en el certificado de desarrollo: el navegador bloquea la página. Ejecutar dotnet dev-certs https --trust una vez.
· Usar el puerto equivocado: la salida de dotnet run indica los puertos exactos.
· Olvidar Ctrl+C antes de continuar: el proceso queda en segundo plano y bloquea el siguiente dotnet run.

Verificación: Swagger UI debe mostrar los dos endpoints y permitir ejecutar el GET (que devuelve una lista vacía []).

Paso 17 — Crear el proyecto de tests y una prueba unitaria de la entidad

Acción 1: En el panel Solution Explorer, expandir tests → AceriaApp.Monolito.Tests.
Acción 2: Hacer doble clic en UnitTest1.cs y renombrarlo a OrdenTests.cs.
Acción 3: Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.Dominio;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Monolito.Tests;

public class OrdenTests
{
```

```
    [Fact]
    public void Orden_ConDatosValidos_SeCreaCorrectamente()
    {
        var orden = new Orden
        {
        Id = Guid.NewGuid(),
        Cliente = "ACX-001",
        Material = "CHAPA-3MM",
        Cantidad = 10,
        FechaCreacion = DateTime.UtcNow
        };

     orden.Cliente.Should().Be("ACX-001");
     orden.Cantidad.Should().Be(10);
    }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: En la terminal, escribir cd .... y pulsar Enter para volver a la carpeta del punto, y después escribir dotnet add tests\AceriaApp.Monolito.Tests\AceriaApp.Monolito.Tests.csproj package FluentAssertions y pulsar Enter.

Por qué se hace: la prueba unitaria verifica que la entidad Orden se instancia correctamente con datos válidos. FluentAssertions proporciona una sintaxis de aserción más legible que Assert.

Errores comunes:

· No añadir el paquete FluentAssertions: Should() no se resuelve.
· Usar Assert.Equal en lugar de Should().Be(): el curso usa FluentAssertions como convención.
· Olvidar el using AceriaApp.Monolito.Api.Dominio;: la clase Orden no se resuelve.

Verificación: OrdenTests.cs debe compilar sin errores y mostrar el método Orden_ConDatosValidos_SeCreaCorrectamente.

Paso 18 — Ejecutar los tests

Acción 1: Escribir dotnet test AceriaApp.sln y pulsar Enter.
Acción 2: Verificar que la salida muestra Passed! - Failed: 0, Passed: 1, Skipped: 0.
Acción 3: Verificar que el tiempo total es inferior a 30 segundos.

Por qué se hace: ejecutar los tests verifica que el proyecto de pruebas está correctamente configurado y que la entidad Orden funciona como se espera.

Errores comunes:

· Ejecutar dotnet test sin especificar la solución: puede ejecutar tests de otros proyectos.
· No compilar antes de ejecutar: dotnet test compila por defecto, pero si hay errores de compilación, los tests no se ejecutan.
· Tener la API en ejecución: el puerto puede estar ocupado, aunque dotnet test no arranca la API.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 1, Skipped: 0, Total: 1.

Paso 19 — Crear el ADR de la decisión arquitectónica

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta del punto 1.1-monolito-vs-microservicios.
Acción 2: Seleccionar Add → New Folder y nombrar la carpeta docs.

Acción 3: Hacer clic derecho sobre docs, seleccionar Add → New Item → Text File y nombrar el archivo ADR-001-monolito-vs-microservicios.md.
Acción 4: Escribir exactamente el siguiente contenido:

```
# ADR-001: Elección entre monolito y microservicios

## Estado
Aceptado

## Contexto
El proyecto AceriaApp debe gestionar órdenes de producción, inventario de materias primas y registro de clientes. El equipo inicial es de dos personas y la frecuencia de despliegue prevista es semanal.

## Decisión
Se adopta un monolito modular como punto de partida, organizado por capas (presentación, aplicación, dominio, infraestructura), con la posibilidad de extraer microservicios en módulos posteriores.

## Alternativas consideradas
- Microservicios desde el inicio: descartado por el coste operativo y el tamaño reducido del equipo.
- Monolito sin separación de capas: descartado por la dificultad de evolución futura.

## Consecuencias
- Positivas: simplicidad operativa, transacciones ACID, despliegue único.
- Negativas: acoplamiento del ciclo de vida, escalado homogéneo.
- Mitigación: modularización interna estricta para facilitar la extracción futura de servicios.
```

Acción 5: Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: el ADR documenta la decisión arquitectónica, las alternativas consideradas y las consecuencias. Es un artefacto obligatorio en cada punto del curso.

Errores comunes:

· Omitir la sección Alternativas consideradas: el ADR pierde valor porque no muestra el análisis.
· No incluir las consecuencias negativas: el ADR queda sesgado.
· Guardar el ADR en la raíz del punto en lugar de docs/: rompe la estructura del repositorio.

Verificación: el archivo docs\ADR-001-monolito-vs-microservicios.md debe existir con las cinco secciones.

Paso 20 — Crear el README del punto

Acción 1: En la carpeta del punto, hacer clic derecho y seleccionar Add → New Item → Text File.
Acción 2: Nombrar el archivo README.md.
Acción 3: Escribir exactamente el siguiente contenido:

```
# Punto 1.1 — Arquitectura monolítica vs. microservicios

## Objetivo
Comprender las diferencias entre monolito y microservicios y crear la estructura inicial del proyecto AceriaApp.

## Estado del proyecto
- Solución `AceriaApp.sln` con dos proyectos: `AceriaApp.Monolito.Api` y `AceriaApp.Monolito.Tests`.
- API Minimal con endpoints `GET /api/v1/ordenes` y `POST /api/v1/ordenes`.
- Persistencia con EF Core 8.0.0 y SQL Server Express (`AceriaDb`).
- Documentación OpenAPI con Swagger.
- Un test unitario de la entidad `Orden`.

## Cómo ejecutarlo
```

1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Ejecutar `dotnet ef database update` en `src\AceriaApp.Monolito.Api`.
4. Ejecutar `dotnet run` en `src\AceriaApp.Monolito.Api`.
5. Navegar a `https://localhost:7xxx/swagger`.

Qué se ha añadido respecto al punto anterior
Este es el primer punto del curso. No hay punto anterior.

Documentación
- [ADR-001](docs/ADR-001-monolito-vs-microservicios.md)

Acción 4: Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: el README explica el objetivo, el estado, la ejecución y los añadidos del punto. Es obligatorio en cada punto del curso.

Errores comunes:

· Omitir la sección Qué se ha añadido respecto al punto anterior: el alumno pierde la trazabilidad.
· No enlazar el ADR: el README queda incompleto.
· Escribir el README en la raíz del repositorio en lugar de la carpeta del punto.

Verificación: el archivo README.md debe existir en la carpeta del punto con las cinco secciones.

Paso 21 — Compilar la solución completa

Acción 1: Escribir dotnet build AceriaApp.sln y pulsar Enter.
Acción 2: Verificar que la salida muestra Build succeeded. y 0 Warning(s), 0 Error(s).
Acción 3: Verificar que el tiempo de compilación es inferior a 60 segundos.

Por qué se hace: compilar la solución completa verifica que todos los proyectos se resuelven correctamente y que no hay errores de dependencias.

Errores comunes:

· Tener la API en ejecución: el archivo .dll está bloqueado y la compilación falla.
· No restaurar paquetes: dotnet build restaura por defecto, pero si hay un error de red, hay que ejecutar dotnet restore primero.
· Usar una versión distinta del SDK: dotnet --version debe devolver 8.x.x.

Verificación: la salida debe mostrar Build succeeded. con 0 Error(s).

Paso 22 — Ejecutar todos los tests de la solución

Acción 1: Escribir dotnet test AceriaApp.sln --no-build y pulsar Enter.
Acción 2: Verificar que la salida muestra Passed! - Failed: 0, Passed: 1, Skipped: 0.
Acción 3: Verificar que el tiempo total es inferior a 30 segundos.

Por qué se hace: ejecutar todos los tests verifica que la solución compila y que las pruebas pasan. --no-build reutiliza la compilación anterior para ahorrar tiempo.

Errores comunes:

· Ejecutar dotnet test sin --no-build después de dotnet build: recompila innecesariamente.
· No haber compilado antes: --no-build falla si no hay artefactos.
· Tener tests de otros puntos en la solución: la solución solo contiene los proyectos de este punto.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 1, Skipped: 0, Total: 1.

Paso 23 — Crear el .gitignore de la solución

Acción 1: En la carpeta del punto, hacer clic derecho y seleccionar Add → New Item → Text File.
Acción 2: Nombrar el archivo .gitignore.
Acción 3: Escribir exactamente el siguiente contenido:

```
bin/
obj/
*.user
*.suo
.vs/
TestResults/
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que el archivo aparece en la carpeta del punto.

Por qué se hace: el .gitignore evita que los artefactos de compilación y los archivos de usuario de Visual Studio se incluyan en el control de versiones.

Errores comunes:

· Olvidar bin/ y obj/: el repositorio se llena de artefactos.
· Escribir bin sin la barra final: el patrón no coincide con las carpetas dentro de los proyectos.
· Escribir el .gitignore en la raíz del repositorio en lugar de la carpeta del punto: afecta a todos los puntos.

Verificación: git status no debe mostrar archivos dentro de bin/ ni obj/.

Paso 24 — Inicializar Git, hacer el commit inicial y verificar el estado final

Acción 1: Escribir cd .... y pulsar Enter para volver a la raíz del repositorio AceriaApp.
Acción 2: Escribir git init y pulsar Enter (si no está inicializado).
Acción 3: Escribir git add . y pulsar Enter.
Acción 4: Escribir git commit -m "Punto 1.1: estructura inicial del proyecto AceriaApp" y pulsar Enter.
Acción 5: Escribir git log --oneline y pulsar Enter para verificar el commit y git status para verificar que el árbol está limpio.

Por qué se hace: el commit inicial registra el estado del punto 1.1 en el historial de Git. Cada punto del curso tendrá su propio commit para facilitar la comparación.

Errores comunes:

· No tener configurado user.name y user.email: git commit falla. Ejecutar git config --global user.name "Nombre" y git config --global user.email "correo@dominio.com".
· Añadir la carpeta bin/ y obj/ al commit: el .gitignore del paso 23 lo evita.
· Hacer commit antes de git add .: no hay cambios en el índice.

Verificación: git log --oneline debe mostrar un commit con el mensaje Punto 1.1: estructura inicial del proyecto AceriaApp. git status debe mostrar nothing to commit, working tree clean.

Errores comunes del ejercicio completo

Error Causa Solución
dotnet ef no se reconoce La herramienta dotnet-ef no está instalada globalmente Ejecutar dotnet tool install --global dotnet-ef --version 8.0.0
Unable to create an object of type 'AceriaDbContext' Falta el paquete Microsoft.EntityFrameworkCore.Design Ejecutar dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.0
A network-related or instance-specific error SQL Server Express no está arrancado Abrir Services.msc y arrancar SQL Server (SQLEXPRESS)
InvalidOperationException: ConnectionString 'AceriaDb' not found La clave de la cadena de conexión no coincide Verificar que appsettings.json contiene "AceriaDb" exactamente
The name 'UseSqlServer' does not exist Falta el using Microsoft.EntityFrameworkCore; Añadir el using al principio de Program.cs
Swagger no aparece El entorno no es Development Verificar ASPNETCORE_ENVIRONMENT=Development o ejecutar dotnet run --environment Development
El POST devuelve 400 El cuerpo JSON no coincide con la entidad Orden Verificar que el JSON tiene cliente, material, cantidad
El GET devuelve 500 La base de datos no existe Ejecutar dotnet ef database update
dotnet test no encuentra tests El proyecto de tests no está en la solución Ejecutar dotnet sln add tests\AceriaApp.Monolito.Tests\AceriaApp.Monolito.Tests.csproj
El commit de Git falla user.name y user.email no configurados Ejecutar git config --global user.name y git config --global user.email

---

Reto resuelto paso a paso

Reto: añadir un endpoint GET /api/v1/ordenes/{id} que devuelva una orden concreta por su identificador, y un test unitario que verifique que devuelve null cuando la orden no existe.

Paso 1: En Program.cs, añadir después del MapGet existente:

```
app.MapGet("/api/v1/ordenes/{id:guid}", async (Guid id, AceriaDbContext db) =>
{
    var orden = await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(o => o.Id == id);
    return orden is not null ? Results.Ok(orden) : Results.NotFound();
});
```

Paso 2: Explicar línea por línea:

Línea 1: app.MapGet("/api/v1/ordenes/{id:guid}", ...) → define un endpoint GET con un parámetro de ruta id restringido a Guid.
Línea 2: async (Guid id, AceriaDbContext db) => → el parámetro id se enlaza desde la ruta y db se inyecta desde el contenedor.
Línea 3: var orden = await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(o => o.Id == id); → consulta la primera orden cuyo Id coincide, sin tracking.
Línea 4: return orden is not null ? Results.Ok(orden) : Results.NotFound(); → devuelve 200 con la orden o 404 si no existe.

Paso 3: Guardar el archivo con Ctrl+S.

Paso 4: En tests\AceriaApp.Monolito.Tests, añadir un archivo OrdenEndpointsTests.cs con:

```
using AceriaApp.Monolito.Api.Dominio;
using AceriaApp.Monolito.Api.Infraestructura;
using FluentAssertions;
using Microsoft.EntityFrameworkCore;
using Xunit;

namespace AceriaApp.Monolito.Tests;
```

```
public class OrdenEndpointsTests
{
    [Fact]
    public async Task GetOrdenPorId_Inexistente_DevuelveNull()
    {
        var options = new DbContextOptionsBuilder<AceriaDbContext>()
            .UseInMemoryDatabase("TestDb")
            .Options;
        await using var db = new AceriaDbContext(options);
        var id = Guid.NewGuid();

        var orden = await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(o => o.Id == id);

     orden.Should().BeNull();
    }
}
```

Paso 5: Añadir el paquete Microsoft.EntityFrameworkCore.InMemory:

dotnet add tests\AceriaApp.Monolito.Tests\AceriaApp.Monolito.Tests.csproj package Microsoft.EntityFrameworkCore.InMemory --version 8.0.0

Paso 6: Ejecutar dotnet test AceriaApp.sln y verificar que pasan los dos tests.

Paso 7: Verificar en Swagger que aparece el nuevo endpoint GET /api/v1/ordenes/{id}.

Paso 8: Hacer commit con git commit -m "Reto 1.1: endpoint GET por id y test de 404".

Verificación: Swagger debe mostrar tres endpoints y dotnet test debe mostrar Passed: 2.

Resultado esperado

Al finalizar este punto, el alumno debe tener:

· La carpeta raíz C:\cursos\AceriaApp\ con el README.md raíz del repositorio.
· La carpeta modulo-1-introduccion-microservicios/1.1-monolito-vs-microservicios/ con la solución AceriaApp.sln, los proyectos src/AceriaApp.Monolito.Api y tests/AceriaApp.Monolito.Tests, la carpeta docs/ con el ADR-001, el README.md del punto y el .gitignore.
· La API ejecutándose con Swagger mostrando GET /api/v1/ordenes, POST /api/v1/ordenes y (tras el reto) GET /api/v1/ordenes/{id}.
· La tabla dbo.Ordenes creada en SQL Server Express dentro de la base de datos AceriaDb.
· Los tests pasando con dotnet test AceriaApp.sln.
· Un commit inicial en Git con el mensaje Punto 1.1: estructura inicial del proyecto AceriaApp.
· Un archivo .gitignore que excluye bin/, obj/, *.user, *.suo, .vs/ y TestResults/.

Conclusión y enlace al siguiente punto

El punto 1.1 ha establecido los fundamentos conceptuales de la comparación entre monolito y microservicios, y ha creado la estructura raíz del repositorio AceriaApp junto con la estructura inicial del punto. Esta base permite avanzar hacia el punto 1.2, donde se analizarán con detalle las ventajas e inconvenientes de cada estilo arquitectónico y se medirá el impacto de cada decisión sobre el proyecto creciente.

Fin del punto 1.1.

Espero confirmación del usuario antes de continuar con el punto 1.2.

PARTE PRÁCTICA (27 minutos)

Ejercicio: Añadir al punto 1.1 un documento de análisis comparativo de ventajas e inconvenientes en docs/, una clase de cálculo de métricas de acoplamiento en el proyecto monolítico y un test unitario que verifique las métricas. El punto 1.2 parte del contenido del punto 1.1 sin modificar ningún archivo existente: solo añade archivos nuevos.

Ubicación de trabajo: C:\cursos\AceriaApp\modulo-1-introduccion-microservicios\

Paso 1 — Crear la carpeta del punto 1.2 a partir del punto 1.1

Acción 1: Abrir la terminal integrada de Visual Studio 2026 con Ctrl+ñ (o Ctrl+`` en teclados no españoles).
Acción 2: Escribir cd C:\cursos\AceriaApp\modulo-1-introduccion-microservicios y pulsar Enter.
Acción 3: Escribir mkdir 1.2-ventajas-inconvenientes y pulsar Enter.
Acción 4: Escribir xcopy 1.1-monolito-vs-microservicios 1.2-ventajas-inconvenientes /E /I /H /Y y pulsar Enter.
Acción 5: Escribir dir 1.2-ventajas-inconvenientes y pulsar Enter para verificar que se ha copiado el contenido.

Por qué se hace: cada punto del curso parte del contenido del punto anterior. El punto 1.2 se construye sobre el punto 1.1 copiando su contenido y añadiendo archivos nuevos sin modificar los existentes.

Errores comunes:

· Usar copy en lugar de xcopy: copy no copia carpetas de forma recursiva.
· Olvidar el flag /E: no se copian las subcarpetas vacías.
· Olvidar el flag /I: xcopy pregunta si el destino es un directorio cuando no existe.
· Olvidar el flag /Y: xcopy pide confirmación para sobrescribir.

Verificación: al ejecutar dir 1.2-ventajas-inconvenientes, deben aparecer AceriaApp.sln, README.md, src, tests y docs.

Paso 2 — Abrir la solución copiada en Visual Studio 2026

Acción 1: Escribir cd 1.2-ventajas-inconvenientes y pulsar Enter.
Acción 2: Escribir start AceriaApp.sln y pulsar Enter.
Acción 3: Verificar que Visual Studio 2026 abre la solución y muestra los proyectos heredados del punto 1.1.
Acción 4: Verificar que la solución compila con Ctrl+Shift+B y que la salida muestra Build succeeded..

Por qué se hace: abrir la solución copiada permite trabajar en el punto 1.2 sobre la base del punto 1.1. La compilación verifica que la copia es correcta.

Errores comunes:

· Abrir la solución del punto 1.1 en lugar de la del punto 1.2: los cambios se aplican al punto equivocado.
· No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
· Tener la API del punto 1.1 en ejecución: el puerto queda ocupado y la compilación falla.

Verificación: Visual Studio 2026 muestra la solución AceriaApp con los dos proyectos y la compilación es correcta.

Paso 3 — Crear la carpeta docs del punto 1.2 si no existe

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta del punto 1.2-ventajas-inconvenientes.
Acción 2: Seleccionar Add → New Folder y nombrar la carpeta docs si no existe.
Acción 3: Verificar que la carpeta docs contiene el archivo ADR-001-monolito-vs-microservicios.md copiado del punto 1.1.
Acción 4: Verificar que el archivo ADR-001 no se modifica en este punto.

Por qué se hace: el punto 1.2 parte del contenido del punto 1.1, que ya incluye la carpeta docs con el ADR-001. No se modifica ningún archivo existente.

Errores comunes:

· Crear la carpeta docs si ya existe: mkdir falla con Cannot create a file when that file already exists.
· Modificar el ADR-001 existente: rompe la regla de no modificar archivos del punto anterior.
· Eliminar el ADR-001 para crear uno nuevo: pierde la trazabilidad del punto 1.1.

Verificación: la carpeta docs existe y contiene el archivo ADR-001-monolito-vs-microservicios.md sin modificaciones.

Paso 4 — Crear el documento de análisis comparativo en docs

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta docs.
Acción 2: Seleccionar Add → New Item → Text File y nombrar el archivo analisis-ventajas-inconvenientes.md.
Acción 3: Escribir exactamente el siguiente contenido:

Análisis de ventajas e inconvenientes — Punto 1.2

Marco de análisis

Cuatro dimensiones: despliegue, escalado, organización y consistencia.
Consideración transversal: coste operativo total.

Ventajas del monolito

Despliegue
- Pipeline único.
- Despliegue atómico.
- Reversión atómica.

Consistencia
- Transacciones ACID sobre una única base de datos.
- Integridad referencial declarativa.
- Consultas con JOIN sin llamadas de red.

Escalado
- Réplica del artefacto completo.
- Configuración idéntica para todas las réplicas.

Organización
- Un único repositorio.
- Un único equipo responsable.
- Decisiones técnicas en una sola conversación.

Inconvenientes del monolito

- Acoplamiento del ciclo de vida.
- Escalado homogéneo.

- Barrera tecnológica.
- Coordinación de equipos creciente.
- Adopción de nuevas tecnologías bloqueada por el conjunto.
- Cuello de botella organizativo a partir de cierto tamaño.

Ventajas de los microservicios

Despliegue
- Pipeline independiente por servicio.
- Frecuencia de despliegue ajustada a cada servicio.
- Riesgo y reversión limitados al servicio afectado.

Escalado
- Selectividad: cada servicio se replica según su carga.
- Coste de infraestructura ajustado a la demanda real.

Organización
- Autonomía de equipos.
- Decisiones técnicas distribuidas.
- Sin conflictos de fusión entre equipos.

Inconvenientes de los microservicios

- Complejidad distribuida.
- Consistencia eventual.
- Observabilidad distribuida.
- Coste operativo por servicio.
- Duplicación de utilidades comunes.
- Gestión de versiones de contratos públicos.
- Latencia de red en cada llamada.

Costes ocultos

Monolito
- Coordinación de equipos.
- Oportunidad tecnológica.
- Escalado homogéneo.

Microservicios
- Complejidad accidental.
- Observabilidad distribuida.
- Duplicación de utilidades.

Criterios de decisión aplicados al proyecto del curso

```
| Criterio | Valor | Decisión |
|----------|-------|----------|
| Tamaño del dominio | Mediano | Monolito modular |
| Número de equipos | 2 | Monolito modular |
| Frecuencia de despliegue | Semanal | Monolito modular |
| Tolerancia al fallo | Baja | Monolito modular |
| Consistencia requerida | Fuerte | Monolito modular |
| Madurez operativa | Inicial | Monolito modular |
```

Conclusión

El proyecto empieza como monolito modular. La evolución hacia microservicios se justificará cuando alguno de los seis criterios cambie.

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que el archivo aparece en la carpeta docs sin subrayados rojos.

Por qué se hace: el documento de análisis comparativo es el artefacto de docs específico del punto 1.2. No modifica el ADR-001 existente ni ningún otro archivo del punto 1.1.

Errores comunes:

· Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
· Modificar el ADR-001 existente: viola la regla de no modificar archivos del punto anterior.
· Guardar el archivo en la raíz del punto en lugar de docs: rompe la estructura del repositorio.

Verificación: el archivo docs\analisis-ventajas-inconvenientes.md debe existir con las nueve secciones.

Paso 5 — Crear la carpeta Metricas en el proyecto monolítico

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.Monolito.Api.
Acción 2: Seleccionar Add → New Folder y nombrar la carpeta Metricas.
Acción 3: Verificar que la carpeta Metricas aparece dentro de AceriaApp.Monolito.Api sin afectar a las carpetas Dominio e Infraestructura.
Acción 4: Verificar que no se ha modificado ningún archivo existente del punto 1.1.

Por qué se hace: la carpeta Metricas albergará la clase de cálculo de métricas de acoplamiento. Se crea como carpeta nueva sin tocar las carpetas existentes.

Errores comunes:

· Crear la carpeta dentro de Dominio o Infraestructura: rompe la separación de responsabilidades.
· Modificar las carpetas existentes: viola la regla de no modificar archivos del punto anterior.
· Crear la carpeta con otro nombre: rompe la convención del curso.

Verificación: la carpeta Metricas existe dentro de AceriaApp.Monolito.Api y las carpetas Dominio e Infraestructura permanecen intactas.

Paso 6 — Crear la clase MetricasAcoplamiento

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta Metricas.
Acción 2: Seleccionar Add → Class y nombrar la clase MetricasAcoplamiento.cs.
Acción 3: Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.Metricas;

public class MetricasAcoplamiento
{
    public int NumeroDeProyectos { get; init; }
    public int NumeroDeReferenciasEntreProyectos { get; init; }
    public int NumeroDeTablasCompartidas { get; init; }

    public double IndiceAcoplamiento()
    {
        if (NumeroDeProyectos == 0)
        {
            return 0;
        }

        return (double)NumeroDeReferenciasEntreProyectos / NumeroDeProyectos;
    }

    public double IndiceComparticionDeDatos()
    {
        if (NumeroDeTablasCompartidas == 0)
        {
            return 0;
        }

        return (double)NumeroDeTablasCompartidas / NumeroDeProyectos;
    }
```

```
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: la clase MetricasAcoplamiento calcula dos indicadores objetivos: el índice de acoplamiento entre proyectos y el índice de compartición de datos. Estos indicadores permiten medir el grado de acoplamiento del monolito y compararlo con el de los microservicios en módulos posteriores.

Errores comunes:

· Olvidar el namespace: la clase no se resuelve desde otros archivos.
· Usar int para los índices: los índices son ratios y deben ser double.
· No comprobar la división por cero: el método lanza DivideByZeroException si el divisor es cero.

Verificación: la clase MetricasAcoplamiento compila sin errores y expone cuatro propiedades y dos métodos.

Paso 7 — Crear la clase MetricasServicio para el análisis por servicio

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta Metricas.
Acción 2: Seleccionar Add → Class y nombrar la clase MetricasServicio.cs.
Acción 3: Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.Metricas;

public class MetricasServicio
{
    public string Nombre { get; init; } = string.Empty;
    public int NumeroDeEndpoints { get; init; }
    public int NumeroDeTablas { get; init; }
    public int NumeroDeDependenciasExternas { get; init; }

    public double IndiceDeComplejidad()
    {
        return NumeroDeEndpoints + NumeroDeTablas + NumeroDeDependenciasExternas;
    }

    public bool EsCandidatoAExtraccion()
    {
        return NumeroDeDependenciasExternas <= 1 && NumeroDeTablas <= 3;
    }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: la clase MetricasServicio permite analizar cada funcionalidad del monolito y decidir si es candidata a extraerse como microservicio. El método EsCandidatoAExtraccion aplica dos criterios objetivos: pocas dependencias externas y pocas tablas propias.

Errores comunes:

· Usar string Nombre { get; set; } sin inicializar: el compilador emite advertencia de nulabilidad.
· Devolver int en IndiceDeComplejidad: el índice es una suma de enteros, pero se expone como double para facilitar comparaciones.
· No documentar los umbrales de EsCandidatoAExtraccion: el método devuelve un bool sin explicar los criterios.

Verificación: la clase MetricasServicio compila sin errores y expone cuatro propiedades y dos métodos.

Paso 8 — Crear la clase AnalizadorMonolito

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta Metricas.
Acción 2: Seleccionar Add → Class y nombrar la clase AnalizadorMonolito.cs.
Acción 3: Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.Metricas;

public class AnalizadorMonolito
{
    private readonly List<MetricasServicio> _servicios;

    public AnalizadorMonolito(List<MetricasServicio> servicios)
    {
     _servicios = servicios;
    }

    public MetricasAcoplamiento CalcularAcoplamientoGlobal()
    {
        var numeroDeProyectos = _servicios.Count;
        var numeroDeReferencias = _servicios.Sum(s => s.NumeroDeDependenciasExternas);
        var numeroDeTablasCompartidas = _servicios.Sum(s => s.NumeroDeTablas);

        return new MetricasAcoplamiento
        {
        NumeroDeProyectos = numeroDeProyectos,
        NumeroDeReferenciasEntreProyectos = numeroDeReferencias,
        NumeroDeTablasCompartidas = numeroDeTablasCompartidas
        };
    }

    public IEnumerable<MetricasServicio> ServiciosCandidatosAExtraccion()
    {
        return _servicios.Where(s => s.EsCandidatoAExtraccion());
    }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: la clase AnalizadorMonolito agrega las métricas de todos los servicios del monolito y calcula el acoplamiento global. También filtra los servicios que son candidatos a extracción como microservicios.

Errores comunes:

· No inicializar _servicios en el constructor: la clase lanza NullReferenceException al usarla.
· Usar List<MetricasServicio> sin using System.Collections.Generic: el tipo no se resuelve en proyectos con ImplicitUsings desactivado.
· Devolver IEnumerable sin using System.Linq: los métodos Sum y Where no se resuelven.

Verificación: la clase AnalizadorMonolito compila sin errores y expone un constructor y dos métodos.

Paso 9 — Añadir el using necesario en AnalizadorMonolito

Acción 1: En el panel Solution Explorer, hacer doble clic en AnalizadorMonolito.cs.
Acción 2: Añadir la línea using System.Linq; al principio del archivo si no está presente.
Acción 3: Verificar que no hay subrayados rojos en Sum ni en Where.
Acción 4: Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: los métodos Sum y Where pertenecen a System.Linq. En proyectos con ImplicitUsings desactivado, hay que añadir el using explícito.

Errores comunes:

· Añadir using System.Collections.Generic; en lugar de System.Linq: no resuelve los métodos de consulta.
· Añadir el using después del namespace: el compilador no lo reconoce.
· Olvidar el using y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: AnalizadorMonolito.cs compila sin errores y no muestra subrayados rojos.

Paso 10 — Crear el proyecto de tests del punto 1.2

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta tests.
Acción 2: Seleccionar Add → New Project.
Acción 3: Seleccionar la plantilla xUnit Test Project, nombrar el proyecto AceriaApp.Metricas.Tests y pulsar Create.
Acción 4: Verificar que el proyecto aparece en la solución y que se ha creado la carpeta tests\AceriaApp.Metricas.Tests.

Por qué se hace: el punto 1.2 añade un proyecto de tests específico para las métricas. Se crea como proyecto nuevo sin modificar el proyecto de tests existente AceriaApp.Monolito.Tests del punto 1.1.

Errores comunes:

· Crear el proyecto dentro de src en lugar de tests: rompe la estructura.
· Modificar el proyecto de tests del punto 1.1: viola la regla de no modificar archivos del punto anterior.
· Usar dotnet new mstest: el curso usa xUnit.

Verificación: el proyecto AceriaApp.Metricas.Tests aparece en la solución y su carpeta existe en tests\AceriaApp.Metricas.Tests.

Paso 11 — Añadir la referencia del proyecto de tests a la API monolítica

Acción 1: En la terminal, escribir dotnet add tests\AceriaApp.Metricas.Tests\AceriaApp.Metricas.Tests.csproj reference src\AceriaApp.Monolito.Api\AceriaApp.Monolito.Api.csproj y pulsar Enter.
Acción 2: Verificar que la salida muestra Reference added successfully.
Acción 3: Verificar que el archivo AceriaApp.Metricas.Tests.csproj contiene la referencia.

Por qué se hace: la referencia permite que el proyecto de tests acceda a las clases de Metricas del proyecto monolítico. Sin esta referencia, los tests no pueden instanciar MetricasAcoplamiento, MetricasServicio ni AnalizadorMonolito.

Errores comunes:

· Añadir la referencia en dirección inversa: crea una dependencia circular.
· Olvidar el nombre completo del archivo .csproj: el comando falla.
· Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

Verificación: el archivo AceriaApp.Metricas.Tests.csproj contiene un ProjectReference al proyecto monolítico.

Paso 12 — Añadir el paquete FluentAssertions al proyecto de tests

Acción 1: Escribir dotnet add tests\AceriaApp.Metricas.Tests\AceriaApp.Metricas.Tests.csproj package FluentAssertions y pulsar Enter.
Acción 2: Escribir dotnet list tests\AceriaApp.Metricas.Tests\AceriaApp.Metricas.Tests.csproj package y pulsar Enter.
Acción 3: Verificar que la salida muestra FluentAssertions con su versión.

Por qué se hace: FluentAssertions proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar el proyecto de tests del punto 1.1.

Errores comunes:

· Instalar una versión preliminar: el curso usa la última estable.
· Olvidar el paquete: Should() no se resuelve en los tests.
· Añadir el paquete al proyecto equivocado: los tests no compilan.

Verificación: la salida de dotnet list package muestra FluentAssertions con una versión estable.

Paso 13 — Crear el archivo MetricasAcoplamientoTests.cs

Acción 1: En el panel Solution Explorer, expandir tests → AceriaApp.Metricas.Tests.
Acción 2: Hacer doble clic en UnitTest1.cs y renombrarlo a MetricasAcoplamientoTests.cs.
Acción 3: Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.Metricas;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Metricas.Tests;

public class MetricasAcoplamientoTests
{
    [Fact]
    public void IndiceAcoplamiento_SinProyectos_DevuelveCero()
    {
        var metricas = new MetricasAcoplamiento { NumeroDeProyectos = 0 };

     metricas.IndiceAcoplamiento().Should().Be(0);
    }

    [Fact]
    public void IndiceAcoplamiento_ConProyectos_DevuelveRatio()
    {
        var metricas = new MetricasAcoplamiento
        {
       NumeroDeProyectos = 4,
       NumeroDeReferenciasEntreProyectos = 8
        };

     metricas.IndiceAcoplamiento().Should().Be(2.0);
    }

    [Fact]
    public void IndiceComparticionDeDatos_SinTablas_DevuelveCero()
    {
        var metricas = new MetricasAcoplamiento { NumeroDeTablasCompartidas = 0 };

     metricas.IndiceComparticionDeDatos().Should().Be(0);
    }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: los tests verifican los dos métodos de MetricasAcoplamiento con casos normales y con casos límite (división por cero).

Errores comunes:

· Olvidar el using AceriaApp.Monolito.Api.Metricas;: la clase no se resuelve.
· Usar Assert.Equal en lugar de Should().Be(): el curso usa FluentAssertions.
· No cubrir el caso de división por cero: el test no detecta el error.

Verificación: el archivo MetricasAcoplamientoTests.cs compila y contiene tres métodos de test.

Paso 14 — Crear el archivo MetricasServicioTests.cs

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.Metricas.Tests.
Acción 2: Seleccionar Add → Class y nombrar la clase MetricasServicioTests.cs.
Acción 3: Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.Metricas;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Metricas.Tests;

public class MetricasServicioTests
{
    [Fact]
    public void EsCandidatoAExtraccion_ConPocasDependencias_DevuelveTrue()
    {
        var metricas = new MetricasServicio
        {
        Nombre = "Inventario",
        NumeroDeEndpoints = 3,
        NumeroDeTablas = 2,
        NumeroDeDependenciasExternas = 1
        };

     metricas.EsCandidatoAExtraccion().Should().BeTrue();
    }

    [Fact]
    public void EsCandidatoAExtraccion_ConMuchasDependencias_DevuelveFalse()
    {
        var metricas = new MetricasServicio
        {
        Nombre = "Ordenes",
        NumeroDeEndpoints = 5,
        NumeroDeTablas = 4,
        NumeroDeDependenciasExternas = 3
        };

     metricas.EsCandidatoAExtraccion().Should().BeFalse();
    }

    [Fact]
    public void IndiceDeComplejidad_SumaDePropiedades()
    {
        var metricas = new MetricasServicio
        {
        NumeroDeEndpoints = 3,
        NumeroDeTablas = 2,
        NumeroDeDependenciasExternas = 1
        };

     metricas.IndiceDeComplejidad().Should().Be(6);
```

```
    }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: los tests verifican la lógica de EsCandidatoAExtraccion con casos que cumplen y no cumplen los umbrales, y verifican que IndiceDeComplejidad suma correctamente las tres propiedades.

Errores comunes:

· No cubrir el caso que supera los umbrales: el test no verifica la lógica de rechazo.
· Olvidar el using AceriaApp.Monolito.Api.Metricas;: la clase no se resuelve.
· Usar Assert.True en lugar de Should().BeTrue(): el curso usa FluentAssertions.

Verificación: el archivo MetricasServicioTests.cs compila y contiene tres métodos de test.

Paso 15 — Crear el archivo AnalizadorMonolitoTests.cs

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.Metricas.Tests.
Acción 2: Seleccionar Add → Class y nombrar la clase AnalizadorMonolitoTests.cs.
Acción 3: Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.Metricas;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Metricas.Tests;

public class AnalizadorMonolitoTests
{
    [Fact]
    public void CalcularAcoplamientoGlobal_ConServicios_AgregaMetricas()
    {
        var servicios = new List<MetricasServicio>
        {
            new() { Nombre = "Ordenes", NumeroDeDependenciasExternas = 2, NumeroDeTablas = 3 },
            new() { Nombre = "Inventario", NumeroDeDependenciasExternas = 1, NumeroDeTablas = 2 }
        };
        var analizador = new AnalizadorMonolito(servicios);

        var resultado = analizador.CalcularAcoplamientoGlobal();

     resultado.NumeroDeProyectos.Should().Be(2);
     resultado.NumeroDeReferenciasEntreProyectos.Should().Be(3);
     resultado.NumeroDeTablasCompartidas.Should().Be(5);
    }

    [Fact]
    public void ServiciosCandidatosAExtraccion_FiltraCorrectamente()
    {
        var servicios = new List<MetricasServicio>
        {
            new() { Nombre = "Ordenes", NumeroDeDependenciasExternas = 3, NumeroDeTablas = 4 },
            new() { Nombre = "Inventario", NumeroDeDependenciasExternas = 1, NumeroDeTablas = 2 }
        };
        var analizador = new AnalizadorMonolito(servicios);

        var candidatos = analizador.ServiciosCandidatosAExtraccion().ToList();

     candidatos.Should().HaveCount(1);
     candidatos[0].Nombre.Should().Be("Inventario");
    }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.

Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: los tests verifican que AnalizadorMonolito agrega correctamente las métricas y filtra los servicios candidatos a extracción.

Errores comunes:

· Olvidar using System.Collections.Generic;: List<T> no se resuelve.
· Olvidar using System.Linq;: ToList() no se resuelve.
· No comprobar el orden de los candidatos: el test podría pasar por casualidad.

Verificación: el archivo AnalizadorMonolitoTests.cs compila y contiene dos métodos de test.

Paso 16 — Compilar la solución completa

Acción 1: Escribir dotnet build AceriaApp.sln y pulsar Enter.
Acción 2: Verificar que la salida muestra Build succeeded. y 0 Warning(s), 0 Error(s).
Acción 3: Verificar que el tiempo de compilación es inferior a 90 segundos.

Por qué se hace: compilar la solución completa verifica que todos los proyectos, incluidos los nuevos, se resuelven correctamente.

Errores comunes:

· Tener la API en ejecución: el archivo .dll está bloqueado y la compilación falla.
· No restaurar paquetes: dotnet build restaura por defecto, pero si hay un error de red, hay que ejecutar dotnet restore primero.
· Usar una versión distinta del SDK: dotnet --version debe devolver 8.x.x.

Verificación: la salida debe mostrar Build succeeded. con 0 Error(s).

Paso 17 — Ejecutar todos los tests del punto 1.2

Acción 1: Escribir dotnet test tests\AceriaApp.Metricas.Tests\AceriaApp.Metricas.Tests.csproj y pulsar Enter.
Acción 2: Verificar que la salida muestra Passed! - Failed: 0, Passed: 8, Skipped: 0.
Acción 3: Verificar que el tiempo total es inferior a 30 segundos.

Por qué se hace: ejecutar los tests del punto 1.2 verifica que las clases de métricas funcionan correctamente.

Errores comunes:

· Ejecutar dotnet test sin especificar el proyecto: ejecuta también los tests del punto 1.1, lo que alarga el tiempo.
· No compilar antes: dotnet test compila por defecto, pero si hay errores, los tests no se ejecutan.
· Tener la API en ejecución: el puerto puede estar ocupado.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 8, Skipped: 0, Total: 8.

Paso 18 — Ejecutar todos los tests de la solución

Acción 1: Escribir dotnet test AceriaApp.sln --no-build y pulsar Enter.
Acción 2: Verificar que la salida muestra Passed! - Failed: 0, Passed: 9, Skipped: 0.
Acción 3: Verificar que el tiempo total es inferior a 45 segundos.

Por qué se hace: ejecutar todos los tests de la solución verifica que los tests del punto 1.1 y del punto 1.2 pasan conjuntamente.

Errores comunes:

· No haber compilado antes: --no-build falla si no hay artefactos.
· Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
· Modificar los tests del punto 1.1: viola la regla de no modificar archivos del punto anterior.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 9, Skipped: 0, Total: 9.

Paso 19 — Crear el README del punto 1.2

Acción 1: En la carpeta del punto 1.2-ventajas-inconvenientes, hacer clic derecho y seleccionar Add → New Item → Text File.
Acción 2: Nombrar el archivo README.md.
Acción 3: Escribir exactamente el siguiente contenido:

Punto 1.2 — Ventajas e inconvenientes

Objetivo
Analizar las ventajas e inconvenientes de monolito y microservicios con un marco explícito de cuatro dimensiones y añadir al proyecto las métricas de acoplamiento.

Estado del proyecto
- Solución `AceriaApp.sln` con tres proyectos: `AceriaApp.Monolito.Api`, `AceriaApp.Monolito.Tests` y `AceriaApp.Metricas.Tests`.
- Documento `docs/analisis-ventajas-inconvenientes.md` con el análisis comparativo.
- Clases `MetricasAcoplamiento`, `MetricasServicio` y `AnalizadorMonolito` en `src/AceriaApp.Monolito.Api/Metricas/`.
- Ocho tests nuevos en `tests/AceriaApp.Metricas.Tests`.

Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Ejecutar `dotnet ef database update` en `src\AceriaApp.Monolito.Api`.
4. Ejecutar `dotnet run` en `src\AceriaApp.Monolito.Api`.
5. Navegar a `https://localhost:7xxx/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

Qué se ha añadido respecto al punto anterior
- Documento `docs/analisis-ventajas-inconvenientes.md`.
- Carpeta `Metricas` en el proyecto monolítico con tres clases nuevas.
- Proyecto `tests/AceriaApp.Metricas.Tests` con ocho tests nuevos.
- No se ha modificado ningún archivo del punto 1.1.

Documentación
- [ADR-001](docs/ADR-001-monolito-vs-microservicios.md) (heredado del punto 1.1).
- [Análisis de ventajas e inconvenientes](docs/analisis-ventajas-inconvenientes.md).

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que el README del punto 1.1 no se ha modificado.

Por qué se hace: el README del punto 1.2 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 1.1.

Errores comunes:

· Modificar el README del punto 1.1: viola la regla de no modificar archivos del punto anterior.

· Omitir la sección Qué se ha añadido respecto al punto anterior: pierde la trazabilidad.
· Escribir el README en la carpeta del punto 1.1: rompe la separación.

Verificación: el archivo README.md existe en la carpeta del punto 1.2 con las cinco secciones.

Paso 20 — Crear el ADR-002 del punto 1.2

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta docs.
Acción 2: Seleccionar Add → New Item → Text File y nombrar el archivo ADR-002-metricas-acoplamiento.md.
Acción 3: Escribir exactamente el siguiente contenido:

ADR-002: Métricas de acoplamiento para decidir la extracción de servicios

Estado
Aceptado

Contexto
El proyecto AceriaApp ha adoptado un monolito modular. Es necesario decidir qué funcionalidades son candidatas a extraerse como microservicios en módulos posteriores.

Decisión
Se adoptan dos indicadores objetivos: el índice de acoplamiento entre proyectos y el índice de compartición de datos. Una funcionalidad es candidata a extracción si tiene como máximo una dependencia externa y como máximo tres tablas propias.

Alternativas consideradas
- Decidir la extracción por criterio subjetivo: descartado por falta de trazabilidad.
- Extraer todas las funcionalidades a la vez: descartado por el coste operativo.

Consecuencias
- Positivas: decisiones trazables, comparables y medibles.
- Negativas: los umbrales son iniciales y pueden requerir ajuste.
- Mitigación: revisar los umbrales al final de cada módulo.

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que el ADR-001 no se ha modificado.

Por qué se hace: el ADR-002 documenta la decisión de adoptar métricas de acoplamiento. Es un artefacto nuevo del punto 1.2 que no modifica el ADR-001 del punto 1.1.

Errores comunes:

· Modificar el ADR-001: viola la regla de no modificar archivos del punto anterior.
· Omitir la sección Alternativas consideradas: el ADR pierde valor.
· Nombrar el archivo sin el prefijo ADR-002: rompe la convención.

Verificación: el archivo docs\ADR-002-metricas-acoplamiento.md existe con las cinco secciones.

Paso 21 — Verificar que no se ha modificado ningún archivo del punto 1.1

Acción 1: En la terminal, escribir cd C:\cursos\AceriaApp\modulo-1-introduccion-microservicios y pulsar Enter.
Acción 2: Escribir fc /L 1.1-monolito-vs-microservicios\docs\ADR-001-monolito-vs-microservicios.md 1.2-ventajas-inconvenientes\docs\ADR-001-monolito-vs-microservicios.md y pulsar Enter.

Acción 3: Verificar que la salida muestra FC: no differences encountered.
Acción 4: Escribir fc /L 1.1-monolito-vs-microservicios\README.md 1.2-ventajas-inconvenientes\README.md y pulsar Enter.
Acción 5: Verificar que la salida muestra diferencias, porque el README del punto 1.2 es nuevo y describe los añadidos.

Por qué se hace: la comparación con fc verifica que el ADR-001 no se ha modificado y que el README del punto 1.2 es distinto al del punto 1.1.

Errores comunes:

· Usar comp en lugar de fc: comp compara binarios y no muestra diferencias de texto.
· Olvidar el flag /L: fc compara en modo ASCII y muestra líneas largas sin ajuste.
· Comparar archivos de carpetas equivocadas: la comparación no es válida.

Verificación: el ADR-001 no muestra diferencias. El README del punto 1.2 muestra diferencias respecto al del punto 1.1.

Paso 22 — Verificar la estructura del punto 1.2

Acción 1: Escribir tree /F 1.2-ventajas-inconvenientes y pulsar Enter.
Acción 2: Verificar que la salida muestra AceriaApp.sln, README.md, src\AceriaApp.Monolito.Api\Metricas con las tres clases nuevas, tests\AceriaApp.Metricas.Tests con los tres archivos de tests y docs con el ADR-001, el ADR-002 y el análisis comparativo.
Acción 3: Verificar que las carpetas Dominio e Infraestructura del punto 1.1 permanecen intactas.
Acción 4: Verificar que no existe ninguna carpeta labs ni ningún archivo metaprompt.md ni verificacion.md.

Por qué se hace: la verificación estructural confirma que el punto 1.2 añade archivos nuevos sin modificar los existentes.

Errores comunes:

· Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
· Modificar las carpetas del punto 1.1: viola la regla de no modificar.
· Omitir la carpeta Metricas: los tests no encuentran las clases.

Verificación: la estructura contiene los archivos nuevos y conserva los del punto 1.1.

Paso 23 — Añadir el .gitignore al punto 1.2 si no existe

Acción 1: Verificar que el archivo .gitignore existe en la carpeta del punto 1.2 (copiado del punto 1.1).
Acción 2: Si no existe, hacer clic derecho en la carpeta del punto, seleccionar Add → New Item → Text File y nombrar el archivo .gitignore.
Acción 3: Escribir exactamente el siguiente contenido:

```
bin/
obj/
*.user
*.suo
.vs/
TestResults/
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que git status no muestra archivos dentro de bin/ ni obj/.

Por qué se hace: el .gitignore evita que los artefactos de compilación del punto 1.2 se incluyan en el control de versiones. Si ya existe copiado del punto 1.1, no se modifica.

Errores comunes:

· Modificar el .gitignore copiado del punto 1.1: viola la regla de no modificar archivos del punto anterior.
· Olvidar bin/ y obj/: el repositorio se llena de artefactos.
· Crear el .gitignore con otro nombre: el archivo no se reconoce.

Verificación: git status no muestra archivos dentro de bin/ ni obj/.

Paso 24 — Hacer el commit del punto 1.2 y verificar el estado final

Acción 1: Escribir cd C:\cursos\AceriaApp y pulsar Enter.
Acción 2: Escribir git add . y pulsar Enter.
Acción 3: Escribir git commit -m "Punto 1.2: análisis de ventajas e inconvenientes y métricas de acoplamiento" y pulsar Enter.
Acción 4: Escribir git log --oneline y pulsar Enter para verificar los dos commits.
Acción 5: Escribir git status y pulsar Enter para verificar que el árbol está limpio.

Por qué se hace: el commit del punto 1.2 registra el estado acumulado del proyecto. Los dos commits consecutivos permiten comparar el punto 1.1 y el punto 1.2 con git diff.

Errores comunes:

· No tener configurado user.name y user.email: git commit falla.
· Añadir la carpeta bin/ y obj/ al commit: el .gitignore lo evita.
· Hacer commit antes de git add .: no hay cambios en el índice.

Verificación: git log --oneline muestra dos commits: Punto 1.1: estructura inicial del proyecto AceriaApp y Punto 1.2: análisis de ventajas e inconvenientes y métricas de acoplamiento. git status muestra nothing to commit, working tree clean.

Errores comunes del ejercicio completo

Error Causa Solución
xcopy no copia subcarpetas Falta el flag /E Ejecutar xcopy origen destino /E /I /H /Y
fc muestra diferencias en el ADR-001 Se ha modificado el archivo del punto 1.1 Restaurar el ADR-001 desde el punto 1.1
DivideByZeroException en IndiceAcoplamiento No se comprueba el divisor Añadir la comprobación if (NumeroDeProyectos == 0)
NullReferenceException en AnalizadorMonolito No se inicializa _servicios Inicializar en el constructor
The name 'Sum' does not exist Falta el using System.Linq; Añadir el using al principio del archivo
The name 'List' does not exist Falta el using System.Collections.Generic; Añadir el using al principio del archivo
Should() no se resuelve Falta el paquete FluentAssertions Ejecutar dotnet add package FluentAssertions
El proyecto de tests no compila Falta la referencia al proyecto monolítico Ejecutar dotnet add reference
El commit de Git incluye bin/ y obj/ Falta el .gitignore Crear el .gitignore con bin/ y obj/
La compilación falla por el .dll bloqueado La API está en ejecución Detener la API con Ctrl+C

Reto resuelto paso a paso

Reto: añadir un método ServicioMenosAcoplado a AnalizadorMonolito que devuelva el servicio con menor índice de complejidad, y un test que verifique que devuelve el servicio correcto cuando hay empate.

Paso 1: En AnalizadorMonolito.cs, añadir el siguiente método después de ServiciosCandidatosAExtraccion:

```
public MetricasServicio? ServicioMenosAcoplado()
{
    if (_servicios.Count == 0)
    {
        return null;
    }

    return _servicios
        .OrderBy(s => s.IndiceDeComplejidad())
        .ThenBy(s => s.Nombre)
        .First();
}
```

Paso 2: Explicar línea por línea:

Línea 1: public MetricasServicio? ServicioMenosAcoplado() → declara el método con retorno anulable.
Línea 2: { → abre el cuerpo del método.
Línea 3: if (_servicios.Count == 0) → comprueba si la lista está vacía.
Línea 4: { → abre el bloque del caso vacío.
Línea 5: return null; → devuelve null si no hay servicios.
Línea 6: } → cierra el bloque del caso vacío.
Línea 7: return _servicios → comienza la consulta LINQ.
Línea 8: .OrderBy(s => s.IndiceDeComplejidad()) → ordena por índice de complejidad ascendente.
Línea 9: .ThenBy(s => s.Nombre) → desempata por nombre ascendente.
Línea 10: .First(); → devuelve el primer elemento.
Línea 11: } → cierra el cuerpo del método.

Paso 3: Guardar el archivo con Ctrl+S.

Paso 4: En AnalizadorMonolitoTests.cs, añadir el siguiente test:

```
[Fact]
public void ServicioMenosAcoplado_ConEmpate_DesempataPorNombre()
{
    var servicios = new List<MetricasServicio>
    {
        new() { Nombre = "Ordenes", NumeroDeEndpoints = 1, NumeroDeTablas = 1,
NumeroDeDependenciasExternas = 1 },
        new() { Nombre = "Inventario", NumeroDeEndpoints = 1, NumeroDeTablas = 1,
NumeroDeDependenciasExternas = 1 }
    };
    var analizador = new AnalizadorMonolito(servicios);

    var resultado = analizador.ServicioMenosAcoplado();

   resultado.Should().NotBeNull();
   resultado!.Nombre.Should().Be("Inventario");
}
```

Paso 5: Explicar línea por línea:

Línea 1: [Fact] → marca el método como test de xUnit.
Línea 2: public void ServicioMenosAcoplado_ConEmpate_DesempataPorNombre() → declara el test.
Línea 3: { → abre el cuerpo del test.
Línea 4: var servicios = new List<MetricasServicio> → crea la lista de servicios.
Línea 5: { → abre el inicializador de la lista.
Línea 6: new() { Nombre = "Ordenes", ... } → primer servicio con índice de complejidad 3.

Línea 7: new() { Nombre = "Inventario", ... } → segundo servicio con el mismo índice.
Línea 8: }; → cierra el inicializador.
Línea 9: var analizador = new AnalizadorMonolito(servicios); → instancia el analizador.
Línea 10: var resultado = analizador.ServicioMenosAcoplado(); → invoca el método.
Línea 11: resultado.Should().NotBeNull(); → verifica que no es nulo.
Línea 12: resultado!.Nombre.Should().Be("Inventario"); → verifica que el desempate es por nombre.
Línea 13: } → cierra el cuerpo del test.

Paso 6: Ejecutar dotnet test tests\AceriaApp.Metricas.Tests\AceriaApp.Metricas.Tests.csproj y verificar que pasan los nueve tests.

Paso 7: Verificar que el método ServicioMenosAcoplado no modifica ningún archivo del punto 1.1.

Paso 8: Hacer commit con git commit -m "Reto 1.2: método ServicioMenosAcoplado con desempate por nombre".

Verificación: la salida de dotnet test muestra Passed: 9.

Resultado esperado

Al finalizar este punto, el alumno debe tener:

· La carpeta modulo-1-introduccion-microservicios/1.2-ventajas-inconvenientes/ con la solución AceriaApp.sln que contiene tres proyectos.
· El documento docs/analisis-ventajas-inconvenientes.md con el análisis comparativo completo.
· El documento docs/ADR-002-metricas-acoplamiento.md con la decisión sobre las métricas.
· Las clases MetricasAcoplamiento, MetricasServicio y AnalizadorMonolito en src/AceriaApp.Monolito.Api/Metricas/.
· El proyecto tests/AceriaApp.Metricas.Tests con ocho tests (nueve tras el reto).
· Los tests del punto 1.1 y del punto 1.2 pasando conjuntamente con dotnet test AceriaApp.sln.
· El ADR-001 del punto 1.1 sin modificar.
· Dos commits en Git: el del punto 1.1 y el del punto 1.2.

Conclusión y enlace al siguiente punto

El punto 1.2 ha completado el análisis de ventajas e inconvenientes con un marco explícito de cuatro dimensiones y ha añadido al proyecto las métricas de acoplamiento que permitirán decidir qué funcionalidades son candidatas a extraerse como microservicios. Esta base permite avanzar hacia el punto 1.3, donde se estudiarán los principios de diseño que guían la descomposición de un sistema en servicios autónomos.

Fin del punto 1.2.

Espero confirmación del usuario antes de continuar con el punto 1.3.

PARTE PRÁCTICA (27 minutos)

Ejercicio: Añadir al punto 1.2 un documento de principios de diseño en docs/, una clase de validación de límites de servicio en el proyecto monolítico y un test unitario que verifique la validación. El punto 1.3 parte del contenido del punto 1.2 sin modificar ningún archivo existente: solo añade archivos nuevos.

Ubicación de trabajo: C:\cursos\AceriaApp\modulo-1-introduccion-microservicios\

Paso 1 — Crear la carpeta del punto 1.3 a partir del punto 1.2

Acción 1: Abrir la terminal integrada de Visual Studio 2026 con Ctrl+ñ (o Ctrl+`` en teclados no españoles).
Acción 2: Escribir cd C:\cursos\AceriaApp\modulo-1-introduccion-microservicios y pulsar Enter.
Acción 3: Escribir mkdir 1.3-principios-diseno y pulsar Enter.
Acción 4: Escribir xcopy 1.2-ventajas-inconvenientes 1.3-principios-diseno /E /I /H /Y y pulsar Enter.
Acción 5: Escribir dir 1.3-principios-diseno y pulsar Enter para verificar que se ha copiado el contenido.

Por qué se hace: cada punto del curso parte del contenido del punto anterior. El punto 1.3 se construye sobre el punto 1.2 copiando su contenido y añadiendo archivos nuevos sin modificar los existentes.

Errores comunes:

· Usar copy en lugar de xcopy: copy no copia carpetas de forma recursiva.
· Olvidar el flag /E: no se copian las subcarpetas vacías.
· Olvidar el flag /I: xcopy pregunta si el destino es un directorio cuando no existe.
· Olvidar el flag /Y: xcopy pide confirmación para sobrescribir.

Verificación: al ejecutar dir 1.3-principios-diseno, deben aparecer AceriaApp.sln, README.md, src, tests y docs.

Paso 2 — Abrir la solución copiada en Visual Studio 2026

Acción 1: Escribir cd 1.3-principios-diseno y pulsar Enter.
Acción 2: Escribir start AceriaApp.sln y pulsar Enter.
Acción 3: Verificar que Visual Studio 2026 abre la solución y muestra los proyectos heredados del punto 1.2.
Acción 4: Verificar que la solución compila con Ctrl+Shift+B y que la salida muestra Build succeeded..

Por qué se hace: abrir la solución copiada permite trabajar en el punto 1.3 sobre la base del punto 1.2. La compilación verifica que la copia es correcta.

Errores comunes:

· Abrir la solución del punto 1.2 en lugar de la del punto 1.3: los cambios se aplican al punto equivocado.
· No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
· Tener la API del punto 1.2 en ejecución: el puerto queda ocupado y la compilación falla.

Verificación: Visual Studio 2026 muestra la solución AceriaApp con los tres proyectos heredados y la compilación es correcta.

Paso 3 — Verificar que la carpeta docs contiene los documentos del punto 1.2

Acción 1: En el panel Solution Explorer, expandir la carpeta docs del punto 1.3.
Acción 2: Verificar que contiene ADR-001-monolito-vs-microservicios.md, ADR-002-metricas-acoplamiento.md y analisis-ventajas-inconvenientes.md.
Acción 3: Verificar que ninguno de los tres documentos se ha modificado.

Acción 4: Verificar que la carpeta docs no contiene ningún archivo nuevo del punto 1.3 todavía.

Por qué se hace: el punto 1.3 parte del contenido del punto 1.2, que ya incluye la carpeta docs con tres documentos. No se modifica ningún documento existente.

Errores comunes:

· Modificar el ADR-001 o el ADR-002: viola la regla de no modificar archivos del punto anterior.
· Eliminar algún documento existente: pierde la trazabilidad del punto 1.2.
· Crear la carpeta docs si ya existe: mkdir falla.

Verificación: la carpeta docs contiene los tres documentos del punto 1.2 sin modificaciones.

Paso 4 — Crear el documento de principios de diseño en docs

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta docs.
Acción 2: Seleccionar Add → New Item → Text File y nombrar el archivo principios-diseno.md.
Acción 3: Escribir exactamente el siguiente contenido:

Principios de diseño — Punto 1.3

Categorías de principios

Tres categorías: responsabilidad, ocultación y acoplamiento.

Principio de responsabilidad única

Un servicio debe tener una sola razón para cambiar, identificada con una capacidad de negocio.

Principio de ocultación de información

Un servicio debe exponer solo lo necesario y ocultar el modelo de datos, la lógica interna y la topología.

Principio de acoplamiento débil y cohesión fuerte

Los elementos de un servicio deben estar fuertemente relacionados entre sí y débilmente con otros.

Principio de límite del contexto

Cada servicio debe corresponder a un contexto delimitado del dominio.

Principio de autonomía del servicio

Un servicio debe poder operar de forma independiente en despliegue, datos y fallo.

Principio de contrato explícito

La comunicación entre servicios se realiza a través de contratos públicos, versionados y documentados.

Principio de tolerancia al fallo

Un servicio debe seguir operando cuando una dependencia falla, aunque sea con funcionalidad degradada.

Principio de evolución independiente

Un servicio debe poder evolucionar sin obligar a los demás a cambiar al mismo ritmo.

Aplicación al proyecto del curso

```
| Principio | Aplicación |
|-----------|------------|
| Responsabilidad única | Módulos por capacidad de negocio |
| Ocultación | Cada módulo con su propia carpeta |
| Acoplamiento débil | Interfaces entre módulos |
| Límite del contexto | Contextos delimitados |
| Autonomía | Evolución hacia microservicios |
```

```
| Contrato explícito | DTOs públicos |
| Tolerancia al fallo | Polly |
| Evolución independiente | Versionado de contratos |
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que el archivo aparece en la carpeta docs sin subrayados rojos.

Por qué se hace: el documento de principios de diseño es el artefacto de docs específico del punto 1.3. No modifica los tres documentos existentes del punto 1.2.

Errores comunes:

· Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
· Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
· Guardar el archivo en la raíz del punto en lugar de docs: rompe la estructura del repositorio.

Verificación: el archivo docs\principios-diseno.md existe con las diez secciones.

Paso 5 — Crear la carpeta Diseno en el proyecto monolítico

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.Monolito.Api.
Acción 2: Seleccionar Add → New Folder y nombrar la carpeta Diseno.
Acción 3: Verificar que la carpeta Diseno aparece dentro de AceriaApp.Monolito.Api sin afectar a las carpetas Dominio, Infraestructura y Metricas.
Acción 4: Verificar que no se ha modificado ningún archivo existente del punto 1.2.

Por qué se hace: la carpeta Diseno albergará las clases de validación de límites de servicio. Se crea como carpeta nueva sin tocar las carpetas existentes.

Errores comunes:

· Crear la carpeta dentro de Dominio, Infraestructura o Metricas: rompe la separación de responsabilidades.
· Modificar las carpetas existentes: viola la regla de no modificar archivos del punto anterior.
· Crear la carpeta con otro nombre: rompe la convención del curso.

Verificación: la carpeta Diseno existe dentro de AceriaApp.Monolito.Api y las carpetas Dominio, Infraestructura y Metricas permanecen intactas.

Paso 6 — Crear la clase LimiteServicio

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta Diseno.
Acción 2: Seleccionar Add → Class y nombrar la clase LimiteServicio.cs.
Acción 3: Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.Diseno;

public class LimiteServicio
{
    public string Nombre { get; init; } = string.Empty;
    public string CapacidadDeNegocio { get; init; } = string.Empty;
    public string ContextoDelimitado { get; init; } = string.Empty;
    public int NumeroDeConceptos { get; init; }
    public int NumeroDeDependenciasSalientes { get; init; }
    public bool ExponeEntidadesDeDominio { get; init; }
    public bool ComparteBaseDeDatos { get; init; }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: la clase LimiteServicio modela los atributos que definen un límite de servicio según los principios de diseño. Cada propiedad corresponde a un principio: capacidad de negocio al de responsabilidad única, contexto delimitado al de límite del contexto, número de conceptos al de cohesión, dependencias salientes al de acoplamiento, exposición de entidades al de ocultación y compartición de base de datos al de autonomía.

Errores comunes:

· Olvidar el namespace: la clase no se resuelve desde otros archivos.
· No inicializar las propiedades de tipo string con string.Empty: el compilador emite advertencias de nulabilidad.
· Usar set en lugar de init para las propiedades: permite modificar la instancia después de crearla.

Verificación: la clase LimiteServicio compila sin errores y expone siete propiedades.

Paso 7 — Crear la clase ValidadorLimiteServicio

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta Diseno.
Acción 2: Seleccionar Add → Class y nombrar la clase ValidadorLimiteServicio.cs.
Acción 3: Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.Diseno;

public class ValidadorLimiteServicio
{
    public ResultadoValidacion Validar(LimiteServicio limite)
    {
        var errores = new List<string>();

        if (string.IsNullOrWhiteSpace(limite.CapacidadDeNegocio))
        {
            errores.Add("El servicio no declara una capacidad de negocio.");
        }

        if (string.IsNullOrWhiteSpace(limite.ContextoDelimitado))
        {
            errores.Add("El servicio no declara un contexto delimitado.");
        }

        if (limite.NumeroDeConceptos > 3)
        {
            errores.Add("El servicio maneja más de tres conceptos: cohesión baja.");
        }

        if (limite.NumeroDeDependenciasSalientes > 3)
        {
            errores.Add("El servicio depende de más de tres servicios: acoplamiento alto.");
        }

        if (limite.ExponeEntidadesDeDominio)
        {
            errores.Add("El servicio expone entidades de dominio: viola la ocultación.");
        }

        if (limite.ComparteBaseDeDatos)
        {
            errores.Add("El servicio comparte base de datos: viola la autonomía.");
        }

        return new ResultadoValidacion(errores.Count == 0, errores);
    }
}

public record ResultadoValidacion(bool EsValido, IReadOnlyList<string> Errores);
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: la clase ValidadorLimiteServicio aplica los principios de diseño de forma automática. Recibe un LimiteServicio y devuelve un ResultadoValidacion con la lista de errores detectados. Cada comprobación corresponde a un principio concreto.

Errores comunes:

· Olvidar el using System.Collections.Generic;: List<string> no se resuelve.
· No devolver los errores: el validador no informa de qué principio se ha violado.
· Usar bool en lugar de ResultadoValidacion: pierde la información de los errores.

Verificación: la clase ValidadorLimiteServicio compila sin errores y expone un método Validar y un record auxiliar.

Paso 8 — Añadir el using necesario en ValidadorLimiteServicio

Acción 1: En el panel Solution Explorer, hacer doble clic en ValidadorLimiteServicio.cs.
Acción 2: Añadir la línea using System.Collections.Generic; al principio del archivo si no está presente.
Acción 3: Verificar que no hay subrayados rojos en List<string>.
Acción 4: Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: el tipo List<string> pertenece a System.Collections.Generic. En proyectos con ImplicitUsings desactivado, hay que añadir el using explícito.

Errores comunes:

· Añadir using System.Linq; en lugar de System.Collections.Generic: no resuelve List<string>.
· Añadir el using después del namespace: el compilador no lo reconoce.
· Olvidar el using y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: ValidadorLimiteServicio.cs compila sin errores y no muestra subrayados rojos.

Paso 9 — Crear la clase CatalogoLimites

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta Diseno.
Acción 2: Seleccionar Add → Class y nombrar la clase CatalogoLimites.cs.
Acción 3: Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.Diseno;

public class CatalogoLimites
{
    private readonly List<LimiteServicio> _limites;

    public CatalogoLimites(List<LimiteServicio> limites)
    {
     _limites = limites;
    }

    public IEnumerable<LimiteServicio> LimitesValidos(ValidadorLimiteServicio validador)
    {
        return _limites.Where(l => validador.Validar(l).EsValido);
    }

    public IEnumerable<LimiteServicio> LimitesInvalidos(ValidadorLimiteServicio validador)
    {
        return _limites.Where(l => !validador.Validar(l).EsValido);
    }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: la clase CatalogoLimites agrupa los límites de servicio del proyecto y permite filtrarlos por validez. Es el punto de entrada para auditar los límites del monolito antes de extraer microservicios.

Errores comunes:

· No inicializar _limites en el constructor: la clase lanza NullReferenceException.
· Olvidar using System.Linq;: Where no se resuelve.
· Devolver List<LimiteServicio> en lugar de IEnumerable<LimiteServicio>: expone la implementación interna.

Verificación: la clase CatalogoLimites compila sin errores y expone un constructor y dos métodos.

Paso 10 — Añadir el using necesario en CatalogoLimites

Acción 1: En el panel Solution Explorer, hacer doble clic en CatalogoLimites.cs.
Acción 2: Añadir la línea using System.Linq; al principio del archivo si no está presente.
Acción 3: Verificar que no hay subrayados rojos en Where.
Acción 4: Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: el método Where pertenece a System.Linq. En proyectos con ImplicitUsings desactivado, hay que añadir el using explícito.

Errores comunes:

· Añadir using System.Collections.Generic; en lugar de System.Linq: no resuelve Where.
· Añadir el using después del namespace: el compilador no lo reconoce.
· Olvidar el using y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: CatalogoLimites.cs compila sin errores y no muestra subrayados rojos.

Paso 11 — Crear el proyecto de tests del punto 1.3

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta tests.
Acción 2: Seleccionar Add → New Project.
Acción 3: Seleccionar la plantilla xUnit Test Project, nombrar el proyecto AceriaApp.Diseno.Tests y pulsar Create.
Acción 4: Verificar que el proyecto aparece en la solución y que se ha creado la carpeta tests\AceriaApp.Diseno.Tests.

Por qué se hace: el punto 1.3 añade un proyecto de tests específico para las clases de diseño. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

Errores comunes:

· Crear el proyecto dentro de src en lugar de tests: rompe la estructura.
· Modificar los proyectos de tests del punto 1.2: viola la regla de no modificar archivos del punto anterior.
· Usar dotnet new mstest: el curso usa xUnit.

Verificación: el proyecto AceriaApp.Diseno.Tests aparece en la solución y su carpeta existe en tests\AceriaApp.Diseno.Tests.

Paso 12 — Añadir la referencia del proyecto de tests a la API monolítica

Acción 1: En la terminal, escribir dotnet add tests\AceriaApp.Diseno.Tests\AceriaApp.Diseno.Tests.csproj reference src\AceriaApp.Monolito.Api\AceriaApp.Monolito.Api.csproj y pulsar Enter.
Acción 2: Verificar que la salida muestra Reference added successfully.
Acción 3: Verificar que el archivo AceriaApp.Diseno.Tests.csproj contiene la referencia.

Por qué se hace: la referencia permite que el proyecto de tests acceda a las clases de Diseno del proyecto monolítico. Sin esta referencia, los tests no pueden instanciar LimiteServicio, ValidadorLimiteServicio ni CatalogoLimites.

Errores comunes:

· Añadir la referencia en dirección inversa: crea una dependencia circular.
· Olvidar el nombre completo del archivo .csproj: el comando falla.
· Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

Verificación: el archivo AceriaApp.Diseno.Tests.csproj contiene un ProjectReference al proyecto monolítico.

Paso 13 — Añadir el paquete FluentAssertions al proyecto de tests

Acción 1: Escribir dotnet add tests\AceriaApp.Diseno.Tests\AceriaApp.Diseno.Tests.csproj package FluentAssertions y pulsar Enter.
Acción 2: Escribir dotnet list tests\AceriaApp.Diseno.Tests\AceriaApp.Diseno.Tests.csproj package y pulsar Enter.
Acción 3: Verificar que la salida muestra FluentAssertions con su versión.

Por qué se hace: FluentAssertions proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

Errores comunes:

· Instalar una versión preliminar: el curso usa la última estable.
· Olvidar el paquete: Should() no se resuelve en los tests.
· Añadir el paquete al proyecto equivocado: los tests no compilan.

Verificación: la salida de dotnet list package muestra FluentAssertions con una versión estable.

Paso 14 — Crear el archivo ValidadorLimiteServicioTests.cs

Acción 1: En el panel Solution Explorer, expandir tests → AceriaApp.Diseno.Tests.
Acción 2: Hacer doble clic en UnitTest1.cs y renombrarlo a ValidadorLimiteServicioTests.cs.
Acción 3: Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.Diseno;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Diseno.Tests;

public class ValidadorLimiteServicioTests
{
    private readonly ValidadorLimiteServicio _validador = new();

    [Fact]
```

```
    public void Validar_LimiteCompleto_DevuelveValido()
    {
        var limite = new LimiteServicio
        {
        Nombre = "Ordenes",
        CapacidadDeNegocio = "Gestion de ordenes de produccion",
        ContextoDelimitado = "Produccion",
        NumeroDeConceptos = 2,
        NumeroDeDependenciasSalientes = 1,
        ExponeEntidadesDeDominio = false,
        ComparteBaseDeDatos = false
        };

        var resultado = _validador.Validar(limite);

     resultado.EsValido.Should().BeTrue();
     resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinCapacidadDeNegocio_DevuelveError()
    {
        var limite = new LimiteServicio
        {
        Nombre = "Ordenes",
        CapacidadDeNegocio = string.Empty,
        ContextoDelimitado = "Produccion"
        };

        var resultado = _validador.Validar(limite);

     resultado.EsValido.Should().BeFalse();
     resultado.Errores.Should().Contain(e => e.Contains("capacidad de negocio"));
    }

    [Fact]
    public void Validar_ConMuchosConceptos_DevuelveErrorDeCohesion()
    {
        var limite = new LimiteServicio
        {
        CapacidadDeNegocio = "Gestion multiple",
        ContextoDelimitado = "Produccion",
        NumeroDeConceptos = 5
        };

        var resultado = _validador.Validar(limite);

     resultado.EsValido.Should().BeFalse();
     resultado.Errores.Should().Contain(e => e.Contains("cohesión baja"));
    }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: los tests verifican que el validador acepta un límite completo y rechaza los límites que violan los principios de responsabilidad única, cohesión y contexto delimitado.

Errores comunes:

- Olvidar el using AceriaApp.Monolito.Api.Diseno;: las clases no se resuelven.
- Usar Assert.True en lugar de Should().BeTrue(): el curso usa FluentAssertions.
- No cubrir el caso de límite válido: el test solo verifica los errores.

Verificación: el archivo ValidadorLimiteServicioTests.cs compila y contiene tres métodos de test.

Paso 15 — Crear el archivo CatalogoLimitesTests.cs

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.Diseno.Tests.
Acción 2: Seleccionar Add → Class y nombrar la clase CatalogoLimitesTests.cs.
Acción 3: Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.Diseno;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Diseno.Tests;

public class CatalogoLimitesTests
{
    [Fact]
    public void LimitesValidos_FiltraCorrectamente()
    {
        var limites = new List<LimiteServicio>
        {
            new()
            {
          Nombre = "Ordenes",
          CapacidadDeNegocio = "Gestion de ordenes",
          ContextoDelimitado = "Produccion",
          NumeroDeConceptos = 2,
          NumeroDeDependenciasSalientes = 1
            },
            new()
            {
          Nombre = "Cajon",
          CapacidadDeNegocio = "Multiples",
          ContextoDelimitado = "Mixto",
          NumeroDeConceptos = 5,
          NumeroDeDependenciasSalientes = 5
            }
        };
        var catalogo = new CatalogoLimites(limites);
        var validador = new ValidadorLimiteServicio();

        var validos = catalogo.LimitesValidos(validador).ToList();
        var invalidos = catalogo.LimitesInvalidos(validador).ToList();

     validos.Should().HaveCount(1);
     validos[0].Nombre.Should().Be("Ordenes");
     invalidos.Should().HaveCount(1);
     invalidos[0].Nombre.Should().Be("Cajon");
    }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: el test verifica que el catálogo filtra correctamente los límites válidos y los inválidos según el validador.

Errores comunes:

- Olvidar using System.Collections.Generic;: List<T> no se resuelve.
- Olvidar using System.Linq;: ToList() no se resuelve.
- No comprobar el nombre del límite válido: el test podría pasar por casualidad.

Verificación: el archivo CatalogoLimitesTests.cs compila y contiene un método de test.

Paso 16 — Compilar la solución completa

Acción 1: Escribir dotnet build AceriaApp.sln y pulsar Enter.
Acción 2: Verificar que la salida muestra Build succeeded. y 0 Warning(s), 0 Error(s).
Acción 3: Verificar que el tiempo de compilación es inferior a 120 segundos.

Por qué se hace: compilar la solución completa verifica que todos los proyectos, incluidos los nuevos, se resuelven correctamente.

Errores comunes:

· Tener la API en ejecución: el archivo .dll está bloqueado y la compilación falla.
· No restaurar paquetes: dotnet build restaura por defecto, pero si hay un error de red, hay que ejecutar dotnet restore primero.
· Usar una versión distinta del SDK: dotnet --version debe devolver 8.x.x.

Verificación: la salida debe mostrar Build succeeded. con 0 Error(s).

Paso 17 — Ejecutar todos los tests del punto 1.3

Acción 1: Escribir dotnet test tests\AceriaApp.Diseno.Tests\AceriaApp.Diseno.Tests.csproj y pulsar Enter.
Acción 2: Verificar que la salida muestra Passed! - Failed: 0, Passed: 4, Skipped: 0.
Acción 3: Verificar que el tiempo total es inferior a 30 segundos.

Por qué se hace: ejecutar los tests del punto 1.3 verifica que las clases de diseño funcionan correctamente.

Errores comunes:

· Ejecutar dotnet test sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
· No compilar antes: dotnet test compila por defecto, pero si hay errores, los tests no se ejecutan.
· Tener la API en ejecución: el puerto puede estar ocupado.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 4, Skipped: 0, Total: 4.

Paso 18 — Ejecutar todos los tests de la solución

Acción 1: Escribir dotnet test AceriaApp.sln --no-build y pulsar Enter.
Acción 2: Verificar que la salida muestra Passed! - Failed: 0, Passed: 13, Skipped: 0.
Acción 3: Verificar que el tiempo total es inferior a 60 segundos.

Por qué se hace: ejecutar todos los tests de la solución verifica que los tests de los puntos 1.1, 1.2 y 1.3 pasan conjuntamente.

Errores comunes:

· No haber compilado antes: --no-build falla si no hay artefactos.
· Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
· Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 13, Skipped: 0, Total: 13.

Paso 19 — Crear el README del punto 1.3

Acción 1: En la carpeta del punto 1.3-principios-diseno, hacer clic derecho y seleccionar Add → New Item → Text File.
Acción 2: Nombrar el archivo README.md.
Acción 3: Escribir exactamente el siguiente contenido:

```
# Punto 1.3 — Principios de diseño

## Objetivo
Comprender los principios de diseño que guían la descomposición de un sistema en servicios autónomos y añadir al proyecto las clases de validación de límites de servicio.

## Estado del proyecto
- Solución `AceriaApp.sln` con cuatro proyectos: `AceriaApp.Monolito.Api`, `AceriaApp.Monolito.Tests`, `AceriaApp.Metricas.Tests` y `AceriaApp.Diseno.Tests`.
- Documento `docs/principios-diseno.md` con los ocho principios y su aplicación al proyecto.
- Clases `LimiteServicio`, `ValidadorLimiteServicio` y `CatalogoLimites` en `src/AceriaApp.Monolito.Api/Diseno/`.
- Cuatro tests nuevos en `tests/AceriaApp.Diseno.Tests`.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Ejecutar `dotnet ef database update` en `src\AceriaApp.Monolito.Api`.
4. Ejecutar `dotnet run` en `src\AceriaApp.Monolito.Api`.
5. Navegar a `https://localhost:7xxx/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/principios-diseno.md`.
- Carpeta `Diseno` en el proyecto monolítico con tres clases nuevas.
- Proyecto `tests/AceriaApp.Diseno.Tests` con cuatro tests nuevos.
- No se ha modificado ningún archivo del punto 1.2.

## Documentación
- [ADR-001](docs/ADR-001-monolito-vs-microservicios.md) (heredado del punto 1.1).
- [ADR-002](docs/ADR-002-metricas-acoplamiento.md) (heredado del punto 1.2).
- [Análisis de ventajas e inconvenientes](docs/analisis-ventajas-inconvenientes.md) (heredado del punto 1.2).
- [Principios de diseño](docs/principios-diseno.md).
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que el README del punto 1.2 no se ha modificado.

Por qué se hace: el README del punto 1.3 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 1.2.

Errores comunes:

- Modificar el README del punto 1.2: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección Qué se ha añadido respecto al punto anterior: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 1.2: rompe la separación.

Verificación: el archivo README.md existe en la carpeta del punto 1.3 con las cinco secciones.

Paso 20 — Crear el ADR-003 del punto 1.3

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta docs.

Acción 2: Seleccionar Add → New Item → Text File y nombrar el archivo ADR-003-principios-diseno.md.
Acción 3: Escribir exactamente el siguiente contenido:

ADR-003: Aplicación de principios de diseño a los límites de servicio

Estado
Aceptado

Contexto
El proyecto AceriaApp ha adoptado un monolito modular y ha definido métricas de acoplamiento. Es necesario aplicar principios de diseño explícitos para decidir los límites entre módulos y preparar la extracción futura de microservicios.

Decisión
Se adoptan ocho principios: responsabilidad única, ocultación de información, acoplamiento débil y cohesión fuerte, límite del contexto, autonomía, contrato explícito, tolerancia al fallo y evolución independiente. Se implementa un validador automático que audita los límites de servicio del monolito.

Alternativas consideradas
- Aplicar los principios sin validación automática: descartado por falta de trazabilidad.
- Aplicar solo algunos principios: descartado por incoherencia.

Consecuencias
- Positivas: decisiones trazables, auditoría automática, preparación para la extracción.
- Negativas: el validador puede requerir ajuste de umbrales.
- Mitigación: revisar los umbrales al final de cada módulo.

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que los ADR-001 y ADR-002 no se han modificado.

Por qué se hace: el ADR-003 documenta la decisión de aplicar principios de diseño y de validarlos automáticamente. Es un artefacto nuevo del punto 1.3 que no modifica los ADR existentes.

Errores comunes:

· Modificar el ADR-001 o el ADR-002: viola la regla de no modificar archivos del punto anterior.
· Omitir la sección Alternativas consideradas: el ADR pierde valor.
· Nombrar el archivo sin el prefijo ADR-003: rompe la convención.

Verificación: el archivo docs\ADR-003-principios-diseno.md existe con las cinco secciones.

Paso 21 — Verificar que no se ha modificado ningún archivo del punto 1.2

Acción 1: En la terminal, escribir cd C:\cursos\AceriaApp\modulo-1-introduccion-microservicios y pulsar Enter.
Acción 2: Escribir fc /L 1.2-ventajas-inconvenientes\docs\ADR-002-metricas-acoplamiento.md 1.3-principios-diseno\docs\ADR-002-metricas-acoplamiento.md y pulsar Enter.
Acción 3: Verificar que la salida muestra FC: no differences encountered.
Acción 4: Escribir fc /L 1.2-ventajas-inconvenientes\docs\analisis-ventajas-inconvenientes.md 1.3-principios-diseno\docs\analisis-ventajas-inconvenientes.md y pulsar Enter.
Acción 5: Verificar que la salida muestra FC: no differences encountered.

Por qué se hace: la comparación con fc verifica que los documentos del punto 1.2 no se han modificado.

Errores comunes:

· Usar comp en lugar de fc: comp compara binarios y no muestra diferencias de texto.
· Olvidar el flag /L: fc compara en modo ASCII y muestra líneas largas sin ajuste.

· Comparar archivos de carpetas equivocadas: la comparación no es válida.

Verificación: los dos documentos del punto 1.2 no muestran diferencias.

Paso 22 — Verificar la estructura del punto 1.3

Acción 1: Escribir tree /F 1.3-principios-diseno y pulsar Enter.
Acción 2: Verificar que la salida muestra AceriaApp.sln, README.md, src\AceriaApp.Monolito.Api\Diseno con las tres clases nuevas, tests\AceriaApp.Diseno.Tests con los dos archivos de tests y docs con el ADR-001, el ADR-002, el ADR-003, el análisis comparativo y el documento de principios.
Acción 3: Verificar que las carpetas Dominio, Infraestructura y Metricas del punto 1.2 permanecen intactas.
Acción 4: Verificar que no existe ninguna carpeta labs ni ningún archivo metaprompt.md ni verificacion.md.

Por qué se hace: la verificación estructural confirma que el punto 1.3 añade archivos nuevos sin modificar los existentes.

Errores comunes:

· Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
· Modificar las carpetas del punto 1.2: viola la regla de no modificar.
· Omitir la carpeta Diseno: los tests no encuentran las clases.

Verificación: la estructura contiene los archivos nuevos y conserva los del punto 1.2.

Paso 23 — Verificar el .gitignore del punto 1.3

Acción 1: Verificar que el archivo .gitignore existe en la carpeta del punto 1.3 (copiado del punto 1.2).
Acción 2: Abrir el archivo .gitignore con Visual Studio 2026.
Acción 3: Verificar que contiene las líneas bin/, obj/, *.user, *.suo, .vs/ y TestResults/.
Acción 4: Verificar que no se ha modificado respecto al punto 1.2.
Acción 5: Pulsar Ctrl+S si no hay cambios y cerrar el archivo.

Por qué se hace: el .gitignore se hereda del punto 1.2 sin modificaciones. La verificación confirma que no se ha tocado.

Errores comunes:

· Modificar el .gitignore copiado del punto 1.2: viola la regla de no modificar archivos del punto anterior.
· Eliminar alguna línea: el repositorio se llena de artefactos.
· Crear el .gitignore con otro nombre: el archivo no se reconoce.

Verificación: el .gitignore existe y contiene las seis líneas sin modificaciones.

Paso 24 — Hacer el commit del punto 1.3 y verificar el estado final

Acción 1: Escribir cd C:\cursos\AceriaApp y pulsar Enter.
Acción 2: Escribir git add . y pulsar Enter.
Acción 3: Escribir git commit -m "Punto 1.3: principios de diseño y validación de límites de servicio" y pulsar Enter.
Acción 4: Escribir git log --oneline y pulsar Enter para verificar los tres commits.

Acción 5: Escribir git status y pulsar Enter para verificar que el árbol está limpio.

Por qué se hace: el commit del punto 1.3 registra el estado acumulado del proyecto. Los tres commits consecutivos permiten comparar los puntos 1.1, 1.2 y 1.3 con git diff.

Errores comunes:

· No tener configurado user.name y user.email: git commit falla.
· Añadir la carpeta bin/ y obj/ al commit: el .gitignore lo evita.
· Hacer commit antes de git add .: no hay cambios en el índice.

Verificación: git log --oneline muestra tres commits: Punto 1.1, Punto 1.2 y Punto 1.3. git status muestra nothing to commit, working tree clean.

Errores comunes del ejercicio completo

Error Causa Solución
xcopy no copia subcarpetas Falta el flag /E Ejecutar xcopy origen destino /E /I /H /Y
List<string> no se resuelve Falta el using System.Collections.Generic; Añadir el using al principio del archivo
Where no se resuelve Falta el using System.Linq; Añadir el using al principio del archivo
NullReferenceException en CatalogoLimites No se inicializa _limites Inicializar en el constructor
El validador no detecta errores Falta alguna comprobación Revisar las seis comprobaciones del método Validar
El test de límite válido falla El límite tiene más de tres conceptos o más de tres dependencias Ajustar el límite de prueba
Should() no se resuelve Falta el paquete FluentAssertions Ejecutar dotnet add package FluentAssertions
El proyecto de tests no compila Falta la referencia al proyecto monolítico Ejecutar dotnet add reference
La compilación falla por el .dll bloqueado La API está en ejecución Detener la API con Ctrl+C
El commit incluye bin/ y obj/ Falta el .gitignore Verificar que el .gitignore está presente

---

Reto resuelto paso a paso

Reto: añadir un método Resumen a CatalogoLimites que devuelva el número de límites válidos, el número de límites inválidos y el porcentaje de validez, y un test que verifique el cálculo cuando hay tres límites válidos y uno inválido.

Paso 1: En CatalogoLimites.cs, añadir el siguiente record al final del archivo:

```
public record ResumenLimites(int Validos, int Invalidos, double PorcentajeValidez);
```

Paso 2: Explicar línea por línea:

Línea 1: public record ResumenLimites(→ declara un contrato inmutable para el resumen.
Línea 2: int Validos, → número de límites válidos.
Línea 3: int Invalidos, → número de límites inválidos.
Línea 4: double PorcentajeValidez); → porcentaje de validez sobre el total.

Paso 3: En CatalogoLimites.cs, añadir el siguiente método dentro de la clase:

```
public ResumenLimites Resumen(ValidadorLimiteServicio validador)
{
    var validos = _limites.Count(l => validador.Validar(l).EsValido);
    var invalidos = _limites.Count - validos;
    var total = _limites.Count;
    var porcentaje = total == 0 ? 0 : (double)validos / total * 100;
    return new ResumenLimites(validos, invalidos, porcentaje);
}
```

Paso 4: Explicar línea por línea:

Línea 1: public ResumenLimites Resumen(ValidadorLimiteServicio validador) → declara el método.
Línea 2: { → abre el cuerpo del método.
Línea 3: var validos = _limites.Count(l => validador.Validar(l).EsValido); → cuenta los válidos.
Línea 4: var invalidos = _limites.Count - validos; → calcula los inválidos por diferencia.
Línea 5: var total = _limites.Count; → obtiene el total.
Línea 6: var porcentaje = total == 0 ? 0 : (double)validos / total * 100; → calcula el porcentaje con protección contra división por cero.
Línea 7: return new ResumenLimites(validos, invalidos, porcentaje); → devuelve el resumen.
Línea 8: } → cierra el cuerpo del método.

Paso 5: Guardar el archivo con Ctrl+S.

Paso 6: En CatalogoLimitesTests.cs, añadir el siguiente test:

```
[Fact]
public void Resumen_ConTresValidosYUnInvalido_CalculaPorcentaje()
{
    var limites = new List<LimiteServicio>
    {
        new() { CapacidadDeNegocio = "A", ContextoDelimitado = "X", NumeroDeConceptos = 1,
NumeroDeDependenciasSalientes = 1 },
        new() { CapacidadDeNegocio = "B", ContextoDelimitado = "X", NumeroDeConceptos = 1,
NumeroDeDependenciasSalientes = 1 },
        new() { CapacidadDeNegocio = "C", ContextoDelimitado = "X", NumeroDeConceptos = 1,
NumeroDeDependenciasSalientes = 1 },
        new() { CapacidadDeNegocio = string.Empty, ContextoDelimitado = "X" }
    };
    var catalogo = new CatalogoLimites(limites);
    var validador = new ValidadorLimiteServicio();

    var resumen = catalogo.Resumen(validador);

   resumen.Validos.Should().Be(3);
   resumen.Invalidos.Should().Be(1);
   resumen.PorcentajeValidez.Should().Be(75.0);
}
```

Paso 7: Explicar línea por línea:

Línea 1: [Fact] → marca el método como test de xUnit.
Línea 2: public void Resumen_ConTresValidosYUnInvalido_CalculaPorcentaje() → declara el test.
Línea 3: { → abre el cuerpo del test.
Línea 4: var limites = new List<LimiteServicio> → crea la lista.
Línea 5: { → abre el inicializador.
Línea 6: new() { CapacidadDeNegocio = "A", ... } → primer límite válido.
Línea 7: new() { CapacidadDeNegocio = "B", ... } → segundo límite válido.
Línea 8: new() { CapacidadDeNegocio = "C", ... } → tercer límite válido.
Línea 9: new() { CapacidadDeNegocio = string.Empty, ... } → límite inválido por falta de capacidad.
Línea 10: }; → cierra el inicializador.
Línea 11: var catalogo = new CatalogoLimites(limites); → instancia el catálogo.
Línea 12: var validador = new ValidadorLimiteServicio(); → instancia el validador.
Línea 13: var resumen = catalogo.Resumen(validador); → invoca el método.
Línea 14: resumen.Validos.Should().Be(3); → verifica los válidos.
Línea 15: resumen.Invalidos.Should().Be(1); → verifica los inválidos.
Línea 16: resumen.PorcentajeValidez.Should().Be(75.0); → verifica el porcentaje.
Línea 17: } → cierra el cuerpo del test.

Paso 8: Ejecutar dotnet test tests\AceriaApp.Diseno.Tests\AceriaApp.Diseno.Tests.csproj y verificar que pasan los cinco tests.

Paso 9: Verificar que el método Resumen no modifica ningún archivo del punto 1.2.

Paso 10: Hacer commit con git commit -m "Reto 1.3: método Resumen en CatalogoLimites con cálculo de porcentaje".

Verificación: la salida de dotnet test muestra Passed: 5.

Resultado esperado

Al finalizar este punto, el alumno debe tener:

· La carpeta modulo-1-introduccion-microservicios/1.3-principios-diseno/ con la solución AceriaApp.sln que contiene cuatro proyectos.
· El documento docs/principios-diseno.md con los ocho principios y su aplicación al proyecto.
· El documento docs/ADR-003-principios-diseno.md con la decisión sobre la validación automática.
· Las clases LimiteServicio, ValidadorLimiteServicio y CatalogoLimites en src/AceriaApp.Monolito.Api/Diseno/.
· El proyecto tests/AceriaApp.Diseno.Tests con cuatro tests (cinco tras el reto).
· Los tests de los puntos 1.1, 1.2 y 1.3 pasando conjuntamente con dotnet test AceriaApp.sln.
· Los ADR-001 y ADR-002 del punto 1.2 sin modificar.
· Tres commits en Git: el del punto 1.1, el del punto 1.2 y el del punto 1.3.

Conclusión y enlace al siguiente punto

El punto 1.3 ha completado el bloque de fundamentos con los principios de diseño que guían la descomposición de un sistema en servicios autónomos. El proyecto cuenta ahora con métricas de acoplamiento y con un validador automático de límites de servicio que aplica los principios de responsabilidad, ocultación, cohesión, acoplamiento, contexto, autonomía, contrato y tolerancia. Esta base permite avanzar hacia el punto 1.4, donde se estudiará la organización concreta de una solución basada en microservicios: estructura de carpetas, convenciones de nombres y separación entre servicios.

Fin del punto 1.3.

Espero confirmación del usuario antes de continuar con el punto 1.4.

PARTE PRÁCTICA (27 minutos)

Ejercicio: Añadir al punto 1.3 un documento de organización de la solución en docs/, una clase de validación de convenciones de nombres en el proyecto monolítico y un test unitario que verifique la validación. El punto 1.4 parte del contenido del punto 1.3 sin modificar ningún archivo existente: solo añade archivos nuevos.

Ubicación de trabajo: C:\cursos\AceriaApp\modulo-1-introduccion-microservicios\

Paso 1 — Crear la carpeta del punto 1.4 a partir del punto 1.3

Acción 1: Abrir la terminal integrada de Visual Studio 2026 con Ctrl+ñ (o Ctrl+`` en teclados no españoles).
Acción 2: Escribir cd C:\cursos\AceriaApp\modulo-1-introduccion-microservicios y pulsar Enter.

Acción 3: Escribir mkdir 1.4-organizacion-solucion y pulsar Enter.
Acción 4: Escribir xcopy 1.3-principios-diseno 1.4-organizacion-solucion /E /I /H /Y y pulsar Enter.
Acción 5: Escribir dir 1.4-organizacion-solucion y pulsar Enter para verificar que se ha copiado el contenido.

Por qué se hace: cada punto del curso parte del contenido del punto anterior. El punto 1.4 se construye sobre el punto 1.3 copiando su contenido y añadiendo archivos nuevos sin modificar los existentes.

Errores comunes:

· Usar copy en lugar de xcopy: copy no copia carpetas de forma recursiva.
· Olvidar el flag /E: no se copian las subcarpetas vacías.
· Olvidar el flag /I: xcopy pregunta si el destino es un directorio cuando no existe.
· Olvidar el flag /Y: xcopy pide confirmación para sobrescribir.

Verificación: al ejecutar dir 1.4-organizacion-solucion, deben aparecer AceriaApp.sln, README.md, src, tests y docs.

Paso 2 — Abrir la solución copiada en Visual Studio 2026

Acción 1: Escribir cd 1.4-organizacion-solucion y pulsar Enter.
Acción 2: Escribir start AceriaApp.sln y pulsar Enter.
Acción 3: Verificar que Visual Studio 2026 abre la solución y muestra los proyectos heredados del punto 1.3.
Acción 4: Verificar que la solución compila con Ctrl+Shift+B y que la salida muestra Build succeeded..

Por qué se hace: abrir la solución copiada permite trabajar en el punto 1.4 sobre la base del punto 1.3. La compilación verifica que la copia es correcta.

Errores comunes:

· Abrir la solución del punto 1.3 en lugar de la del punto 1.4: los cambios se aplican al punto equivocado.
· No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
· Tener la API del punto 1.3 en ejecución: el puerto queda ocupado y la compilación falla.

Verificación: Visual Studio 2026 muestra la solución AceriaApp con los cuatro proyectos heredados y la compilación es correcta.

Paso 3 — Verificar que la carpeta docs contiene los documentos del punto 1.3

Acción 1: En el panel Solution Explorer, expandir la carpeta docs del punto 1.4.
Acción 2: Verificar que contiene ADR-001, ADR-002, ADR-003, analisis-ventajas-inconvenientes.md y principios-diseno.md.
Acción 3: Verificar que ninguno de los cinco documentos se ha modificado.
Acción 4: Verificar que la carpeta docs no contiene ningún archivo nuevo del punto 1.4 todavía.

Por qué se hace: el punto 1.4 parte del contenido del punto 1.3, que ya incluye la carpeta docs con cinco documentos. No se modifica ningún documento existente.

Errores comunes:

· Modificar alguno de los ADR o documentos existentes: viola la regla de no modificar archivos del punto anterior.
· Eliminar algún documento existente: pierde la trazabilidad del punto 1.3.

· Crear la carpeta docs si ya existe: mkdir falla.

Verificación: la carpeta docs contiene los cinco documentos del punto 1.3 sin modificaciones.

Paso 4 — Crear el documento de organización de la solución en docs

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta docs.
Acción 2: Seleccionar Add → New Item → Text File y nombrar el archivo organizacion-solucion.md.
Acción 3: Escribir exactamente el siguiente contenido:

Organización de la solución — Punto 1.4

Tres decisiones de organización

1. Estructura de repositorios.
2. Estructura de carpetas.
3. Convenciones de nombres.

Estructura de repositorios

Monorepo con una carpeta por módulo y una carpeta por punto.

Estructura de carpetas

Cada punto contiene `src/`, `tests/` y `docs/`.

Convenciones de nombres de proyectos

Patrón: `AceriaApp.Componente.Tipo`.

Tipos: `Api`, `Domain`, `Infrastructure`, `Application`, `Tests`.

Convenciones de espacios de nombres

El espacio de nombres coincide con la estructura de carpetas del proyecto.

Convenciones de commits

Patrón: `Punto N.M: descripción`.

Convenciones de ramas

Prefijos: `feature/`, `fix/`, `docs/`, `refactor/`.

Estructura con varios servicios

Cada servicio tiene su propio conjunto de proyectos y su propio conjunto de tests.

Aplicación al proyecto del curso

```
| Decisión | Aplicación |
|----------|------------|
| Monorepo | `AceriaApp/` con módulos y puntos |
| Carpetas | `src/`, `tests/`, `docs/` |
| Nombres | `AceriaApp.Componente.Tipo` |
| Namespaces | Coinciden con la carpeta |
| Commits | `Punto N.M: descripción` |
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que el archivo aparece en la carpeta docs sin subrayados rojos.

Por qué se hace: el documento de organización de la solución es el artefacto de docs específico del punto 1.4. No modifica los cinco documentos existentes del punto 1.3.

Errores comunes:

· Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.

· Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
· Guardar el archivo en la raíz del punto en lugar de docs: rompe la estructura del repositorio.

Verificación: el archivo docs\organizacion-solucion.md existe con las nueve secciones.

Paso 5 — Crear la carpeta Organizacion en el proyecto monolítico

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.Monolito.Api.
Acción 2: Seleccionar Add → New Folder y nombrar la carpeta Organizacion.
Acción 3: Verificar que la carpeta Organizacion aparece dentro de AceriaApp.Monolito.Api sin afectar a las carpetas Dominio, Infraestructura, Metricas y Diseno.
Acción 4: Verificar que no se ha modificado ningún archivo existente del punto 1.3.

Por qué se hace: la carpeta Organizacion albergará las clases de validación de convenciones de nombres. Se crea como carpeta nueva sin tocar las carpetas existentes.

Errores comunes:

· Crear la carpeta dentro de Dominio, Infraestructura, Metricas o Diseno: rompe la separación de responsabilidades.
· Modificar las carpetas existentes: viola la regla de no modificar archivos del punto anterior.
· Crear la carpeta con otro nombre: rompe la convención del curso.

Verificación: la carpeta Organizacion existe dentro de AceriaApp.Monolito.Api y las carpetas existentes permanecen intactas.

Paso 6 — Crear la clase ConvencionNombres

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta Organizacion.
Acción 2: Seleccionar Add → Class y nombrar la clase ConvencionNombres.cs.
Acción 3: Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.Organizacion;

public class ConvencionNombres
{
    public string Prefijo { get; init; } = "AceriaApp";
    public string Componente { get; init; } = string.Empty;
    public string Tipo { get; init; } = string.Empty;

    public string NombreCompleto()
    {
        return $"{Prefijo}.{Componente}.{Tipo}";
    }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: la clase ConvencionNombres modela la convención de nombres de proyectos y expone un método que devuelve el nombre completo. Es la base para validar que todos los proyectos del curso siguen la convención.

Errores comunes:

· Olvidar el namespace: la clase no se resuelve desde otros archivos.
· No inicializar Prefijo con "AceriaApp": la convención no se aplica por defecto.
· Usar set en lugar de init: permite modificar las propiedades después de crear la instancia.

Verificación: la clase ConvencionNombres compila sin errores y expone tres propiedades y un método.

Paso 7 — Crear la clase ValidadorConvencionNombres

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta Organizacion.
Acción 2: Seleccionar Add → Class y nombrar la clase ValidadorConvencionNombres.cs.
Acción 3: Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.Organizacion;

public class ValidadorConvencionNombres
{
    private readonly string _prefijo;
    private readonly IReadOnlyList<string> _tiposPermitidos;

    public ValidadorConvencionNombres(string prefijo, IReadOnlyList<string> tiposPermitidos)
    {
     _prefijo = prefijo;
     _tiposPermitidos = tiposPermitidos;
    }

    public ResultadoConvencion Validar(string nombreProyecto)
    {
        var errores = new List<string>();

        if (!nombreProyecto.StartsWith(_prefijo + ".", StringComparison.Ordinal))
        {
        errores.Add($"El proyecto no empieza por '{_prefijo}.'.");
        }

        var partes = nombreProyecto.Split('.');
        if (partes.Length != 3)
        {
        errores.Add("El nombre no tiene el formato 'Prefijo.Componente.Tipo'.");
        }
     else if (!_tiposPermitidos.Contains(partes[2]))
        {
        errores.Add($"El tipo '{partes[2]}' no está permitido.");
        }

        return new ResultadoConvencion(errores.Count == 0, errores);
    }
}

public record ResultadoConvencion(bool EsValido, IReadOnlyList<string> Errores);
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: la clase ValidadorConvencionNombres comprueba que un nombre de proyecto cumple el patrón Prefijo.Componente.Tipo y que el tipo pertenece a la lista de tipos permitidos. Es el mecanismo de auditoría de la convención de nombres.

Errores comunes:

- Olvidar el using System.Collections.Generic;: IReadOnlyList<string> y List<string> no se resuelven.
- Usar Contains sin using System.Linq;: el método no se resuelve en listas.
- No comprobar la longitud de las partes: el validador acepta nombres mal formados.

Verificación: la clase ValidadorConvencionNombres compila sin errores y expone un constructor, un método y un record.

Paso 8 — Añadir los using necesarios en ValidadorConvencionNombres

Acción 1: En el panel Solution Explorer, hacer doble clic en ValidadorConvencionNombres.cs.
Acción 2: Añadir las líneas using System.Collections.Generic; y using System.Linq; al principio del archivo si no están presentes.
Acción 3: Verificar que no hay subrayados rojos en IReadOnlyList<string> ni en Contains.
Acción 4: Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: los tipos IReadOnlyList<string> y List<string> pertenecen a System.Collections.Generic. El método Contains pertenece a System.Linq. En proyectos con ImplicitUsings desactivado, hay que añadir los usings explícitos.

Errores comunes:

- Añadir solo System.Collections.Generic y olvidar System.Linq: Contains no se resuelve.
- Añadir los usings después del namespace: el compilador no los reconoce.
- Olvidar los usings y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: ValidadorConvencionNombres.cs compila sin errores y no muestra subrayados rojos.

Paso 9 — Crear la clase CatalogoProyectos

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta Organizacion.
Acción 2: Seleccionar Add → Class y nombrar la clase CatalogoProyectos.cs.
Acción 3: Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.Organizacion;

public class CatalogoProyectos
{
    private readonly List<string> _nombres;

    public CatalogoProyectos(List<string> nombres)
    {
     _nombres = nombres;
    }

    public IEnumerable<string> ProyectosValidos(ValidadorConvencionNombres validador)
    {
        return _nombres.Where(n => validador.Validar(n).EsValido);
    }

    public IEnumerable<string> ProyectosInvalidos(ValidadorConvencionNombres validador)
    {
        return _nombres.Where(n => !validador.Validar(n).EsValido);
    }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: la clase CatalogoProyectos agrupa los nombres de los proyectos del curso y permite filtrarlos por validez según la convención.

Errores comunes:

- No inicializar _nombres en el constructor: la clase lanza NullReferenceException.
- Olvidar using System.Linq;: Where no se resuelve.
- Devolver List<string> en lugar de IEnumerable<string>: expone la implementación interna.

Verificación: la clase CatalogoProyectos compila sin errores y expone un constructor y dos métodos.

Paso 10 — Añadir el using necesario en CatalogoProyectos

Acción 1: En el panel Solution Explorer, hacer doble clic en CatalogoProyectos.cs.
Acción 2: Añadir la línea using System.Linq; al principio del archivo si no está presente.
Acción 3: Verificar que no hay subrayados rojos en Where.
Acción 4: Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: el método Where pertenece a System.Linq. En proyectos con ImplicitUsings desactivado, hay que añadir el using explícito.

Errores comunes:

· Añadir using System.Collections.Generic; en lugar de System.Linq: no resuelve Where.
· Añadir el using después del namespace: el compilador no lo reconoce.
· Olvidar el using y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: CatalogoProyectos.cs compila sin errores y no muestra subrayados rojos.

Paso 11 — Crear el proyecto de tests del punto 1.4

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta tests.
Acción 2: Seleccionar Add → New Project.
Acción 3: Seleccionar la plantilla xUnit Test Project, nombrar el proyecto AceriaApp.Organizacion.Tests y pulsar Create.
Acción 4: Verificar que el proyecto aparece en la solución y que se ha creado la carpeta tests\AceriaApp.Organizacion.Tests.

Por qué se hace: el punto 1.4 añade un proyecto de tests específico para las clases de organización. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

Errores comunes:

· Crear el proyecto dentro de src en lugar de tests: rompe la estructura.
· Modificar los proyectos de tests del punto 1.3: viola la regla de no modificar archivos del punto anterior.
· Usar dotnet new mstest: el curso usa xUnit.

Verificación: el proyecto AceriaApp.Organizacion.Tests aparece en la solución y su carpeta existe en tests\AceriaApp.Organizacion.Tests.

Paso 12 — Añadir la referencia del proyecto de tests a la API monolítica

Acción 1: En la terminal, escribir dotnet add tests\AceriaApp.Organizacion.Tests\AceriaApp.Organizacion.Tests.csproj reference src\AceriaApp.Monolito.Api\AceriaApp.Monolito.Api.csproj y pulsar Enter.
Acción 2: Verificar que la salida muestra Reference added successfully.
Acción 3: Verificar que el archivo AceriaApp.Organizacion.Tests.csproj contiene la referencia.

Por qué se hace: la referencia permite que el proyecto de tests acceda a las clases de Organizacion del proyecto monolítico.

Errores comunes:

· Añadir la referencia en dirección inversa: crea una dependencia circular.
· Olvidar el nombre completo del archivo .csproj: el comando falla.

· Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

Verificación: el archivo AceriaApp.Organizacion.Tests.csproj contiene un ProjectReference al proyecto monolítico.

Paso 13 — Añadir el paquete FluentAssertions al proyecto de tests

Acción 1: Escribir dotnet add tests\AceriaApp.Organizacion.Tests\AceriaApp.Organizacion.Tests.csproj package FluentAssertions y pulsar Enter.
Acción 2: Escribir dotnet list tests\AceriaApp.Organizacion.Tests\AceriaApp.Organizacion.Tests.csproj package y pulsar Enter.
Acción 3: Verificar que la salida muestra FluentAssertions con su versión.

Por qué se hace: FluentAssertions proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

Errores comunes:

· Instalar una versión preliminar: el curso usa la última estable.
· Olvidar el paquete: Should() no se resuelve en los tests.
· Añadir el paquete al proyecto equivocado: los tests no compilan.

Verificación: la salida de dotnet list package muestra FluentAssertions con una versión estable.

Paso 14 — Crear el archivo ValidadorConvencionNombresTests.cs

Acción 1: En el panel Solution Explorer, expandir tests → AceriaApp.Organizacion.Tests.
Acción 2: Hacer doble clic en UnitTest1.cs y renombrarlo a ValidadorConvencionNombresTests.cs.
Acción 3: Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.Organizacion;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Organizacion.Tests;

public class ValidadorConvencionNombresTests
{
    private readonly ValidadorConvencionNombres _validador = new(
     "AceriaApp",
     new[] { "Api", "Domain", "Infrastructure", "Application", "Tests" });

    [Fact]
    public void Validar_NombreCorrecto_DevuelveValido()
    {
        var resultado = _validador.Validar("AceriaApp.Ordenes.Api");

     resultado.EsValido.Should().BeTrue();
     resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinPrefijo_DevuelveError()
    {
        var resultado = _validador.Validar("Ordenes.Api");

     resultado.EsValido.Should().BeFalse();
     resultado.Errores.Should().Contain(e => e.Contains("no empieza por"));
    }

    [Fact]
```

```
    public void Validar_TipoNoPermitido_DevuelveError()
    {
        var resultado = _validador.Validar("AceriaApp.Ordenes.Controlador");
     resultado.EsValido.Should().BeFalse();
     resultado.Errores.Should().Contain(e => e.Contains("no está permitido"));
    }
    [Fact]
    public void Validar_NumeroDePartesIncorrecto_DevuelveError()
    {
        var resultado = _validador.Validar("AceriaApp.Ordenes");
     resultado.EsValido.Should().BeFalse();
     resultado.Errores.Should().Contain(e => e.Contains("Prefijo.Componente.Tipo"));
    }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: los tests verifican que el validador acepta nombres correctos y rechaza nombres sin prefijo, con tipo no permitido y con número de partes incorrecto.

Errores comunes:

- Olvidar el using AceriaApp.Monolito.Api.Organizacion;: las clases no se resuelven.
- Usar Assert.True en lugar de Should().BeTrue(): el curso usa FluentAssertions.
- No cubrir el caso de nombre correcto: el test solo verifica los errores.

Verificación: el archivo ValidadorConvencionNombresTests.cs compila y contiene cuatro métodos de test.

Paso 15 — Crear el archivo CatalogoProyectosTests.cs

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.Organizacion.Tests.
Acción 2: Seleccionar Add → Class y nombrar la clase CatalogoProyectosTests.cs.
Acción 3: Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.Organizacion;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Organizacion.Tests;

public class CatalogoProyectosTests
{
    [Fact]
    public void ProyectosValidos_FiltraCorrectamente()
    {
        var nombres = new List<string>
        {
        "AceriaApp.Ordenes.Api",
        "AceriaApp.Ordenes.Domain",
        "AceriaApp.Ordenes",
        "AceriaApp.Ordenes.Controlador"
        };
        var catalogo = new CatalogoProyectos(nombres);
        var validador = new ValidadorConvencionNombres(
        "AceriaApp",
        new[] { "Api", "Domain", "Infrastructure", "Application", "Tests" });

        var validos = catalogo.ProyectosValidos(validador).ToList();
```

```
        var invalidos = catalogo.ProyectosInvalidos(validador).ToList();

        validos.Should().HaveCount(2);
        validos.Should().Contain("AceriaApp.Ordenes.Api");
        validos.Should().Contain("AceriaApp.Ordenes.Domain");
        invalidos.Should().HaveCount(2);
        invalidos.Should().Contain("AceriaApp.Ordenes");
        invalidos.Should().Contain("AceriaApp.Ordenes.Controlador");
    }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: el test verifica que el catálogo filtra correctamente los nombres válidos y los inválidos según el validador de convenciones.

Errores comunes:

· Olvidar using System.Collections.Generic;: List<T> no se resuelve.
· Olvidar using System.Linq;: ToList() no se resuelve.
· No comprobar los nombres concretos: el test podría pasar por casualidad.

Verificación: el archivo CatalogoProyectosTests.cs compila y contiene un método de test.

Paso 16 — Compilar la solución completa

Acción 1: Escribir dotnet build AceriaApp.sln y pulsar Enter.
Acción 2: Verificar que la salida muestra Build succeeded. y 0 Warning(s), 0 Error(s).
Acción 3: Verificar que el tiempo de compilación es inferior a 150 segundos.

Por qué se hace: compilar la solución completa verifica que todos los proyectos, incluidos los nuevos, se resuelven correctamente.

Errores comunes:

· Tener la API en ejecución: el archivo .dll está bloqueado y la compilación falla.
· No restaurar paquetes: dotnet build restaura por defecto, pero si hay un error de red, hay que ejecutar dotnet restore primero.
· Usar una versión distinta del SDK: dotnet --version debe devolver 8.x.x.

Verificación: la salida debe mostrar Build succeeded. con 0 Error(s).

Paso 17 — Ejecutar todos los tests del punto 1.4

Acción 1: Escribir dotnet test tests\AceriaApp.Organizacion.Tests\AceriaApp.Organizacion.Tests.csproj y pulsar Enter.
Acción 2: Verificar que la salida muestra Passed! - Failed: 0, Passed: 5, Skipped: 0.
Acción 3: Verificar que el tiempo total es inferior a 30 segundos.

Por qué se hace: ejecutar los tests del punto 1.4 verifica que las clases de organización funcionan correctamente.

Errores comunes:

· Ejecutar dotnet test sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
· No compilar antes: dotnet test compila por defecto, pero si hay errores, los tests no se ejecutan.

· Tener la API en ejecución: el puerto puede estar ocupado.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 5, Skipped: 0, Total: 5.

Paso 18 — Ejecutar todos los tests de la solución

Acción 1: Escribir dotnet test AceriaApp.sln --no-build y pulsar Enter.
Acción 2: Verificar que la salida muestra Passed! - Failed: 0, Passed: 18, Skipped: 0.
Acción 3: Verificar que el tiempo total es inferior a 90 segundos.

Por qué se hace: ejecutar todos los tests de la solución verifica que los tests de los puntos 1.1, 1.2, 1.3 y 1.4 pasan conjuntamente.

Errores comunes:

· No haber compilado antes: --no-build falla si no hay artefactos.
· Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
· Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 18, Skipped: 0, Total: 18.

Paso 19 — Crear el README del punto 1.4

Acción 1: En la carpeta del punto 1.4-organizacion-solucion, hacer clic derecho y seleccionar Add → New Item → Text File.
Acción 2: Nombrar el archivo README.md.
Acción 3: Escribir exactamente el siguiente contenido:

Punto 1.4 — Organización de una solución basada en microservicios

Objetivo
Comprender cómo se organiza una solución basada en microservicios y añadir al proyecto las clases de validación de convenciones de nombres.

Estado del proyecto
- Solución `AceriaApp.sln` con cinco proyectos: `AceriaApp.Monolito.Api`, `AceriaApp.Monolito.Tests`, `AceriaApp.Metricas.Tests`, `AceriaApp.Diseno.Tests` y `AceriaApp.Organizacion.Tests`.
- Documento `docs/organizacion-solucion.md` con las tres decisiones de organización y las convenciones.
- Clases `ConvencionNombres`, `ValidadorConvencionNombres` y `CatalogoProyectos` en `src/AceriaApp.Monolito.Api/Organizacion/`.
- Cinco tests nuevos en `tests/AceriaApp.Organizacion.Tests`.

Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Ejecutar `dotnet ef database update` en `src\AceriaApp.Monolito.Api`.
4. Ejecutar `dotnet run` en `src\AceriaApp.Monolito.Api`.
5. Navegar a `https://localhost:7xxx/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

Qué se ha añadido respecto al punto anterior
- Documento `docs/organizacion-solucion.md`.
- Carpeta `Organizacion` en el proyecto monolítico con tres clases nuevas.
- Proyecto `tests/AceriaApp.Organizacion.Tests` con cinco tests nuevos.

- No se ha modificado ningún archivo del punto 1.3.

Documentación
- [ADR-001](docs/ADR-001-monolito-vs-microservicios.md) (heredado del punto 1.1).
- [ADR-002](docs/ADR-002-metricas-acoplamiento.md) (heredado del punto 1.2).
- [ADR-003](docs/ADR-003-principios-diseno.md) (heredado del punto 1.3).
- [Análisis de ventajas e inconvenientes](docs/analisis-ventajas-inconvenientes.md) (heredado del punto 1.2).
- [Principios de diseño](docs/principios-diseno.md) (heredado del punto 1.3).
- [Organización de la solución](docs/organizacion-solucion.md).

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que el README del punto 1.3 no se ha modificado.

Por qué se hace: el README del punto 1.4 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 1.3.

Errores comunes:

· Modificar el README del punto 1.3: viola la regla de no modificar archivos del punto anterior.
· Omitir la sección Qué se ha añadido respecto al punto anterior: pierde la trazabilidad.
· Escribir el README en la carpeta del punto 1.3: rompe la separación.

Verificación: el archivo README.md existe en la carpeta del punto 1.4 con las cinco secciones.

Paso 20 — Crear el ADR-004 del punto 1.4

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta docs.
Acción 2: Seleccionar Add → New Item → Text File y nombrar el archivo ADR-004-organizacion-solucion.md.
Acción 3: Escribir exactamente el siguiente contenido:

ADR-004: Organización de la solución y convenciones de nombres

Estado
Aceptado

Contexto
El proyecto AceriaApp ha adoptado un monolito modular, métricas de acoplamiento y principios de diseño. Es necesario definir la estructura de repositorios, la estructura de carpetas y las convenciones de nombres para preparar la evolución hacia microservicios.

Decisión
Se adopta un monorepo con una carpeta por módulo y una carpeta por punto. Dentro de cada punto se separan `src/`, `tests/` y `docs/`. Los proyectos siguen el patrón `AceriaApp.Componente.Tipo`. Los espacios de nombres coinciden con la estructura de carpetas. Los commits siguen el patrón `Punto N.M: descripción`.

Alternativas consideradas
- Polyrepo por servicio: descartado por el tamaño del equipo y la fase de aprendizaje.
- Estructura plana sin separación de carpetas: descartada por la pérdida de previsibilidad.

Consecuencias
- Positivas: previsibilidad, trazabilidad, preparación para la extracción de servicios.
- Negativas: el monorepo requiere disciplina para mantener la separación.
- Mitigación: validación automática de convenciones con la clase `ValidadorConvencionNombres`.

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que los ADR-001, ADR-002 y ADR-003 no se han modificado.

Por qué se hace: el ADR-004 documenta la decisión de organización y convenciones. Es un artefacto nuevo del punto 1.4 que no modifica los ADR existentes.

Errores comunes:

· Modificar los ADR anteriores: viola la regla de no modificar archivos del punto anterior.
· Omitir la sección Alternativas consideradas: el ADR pierde valor.
· Nombrar el archivo sin el prefijo ADR-004: rompe la convención.

Verificación: el archivo docs\ADR-004-organizacion-solucion.md existe con las cinco secciones.

Paso 21 — Verificar que no se ha modificado ningún archivo del punto 1.3

Acción 1: En la terminal, escribir cd C:\cursos\AceriaApp\modulo-1-introduccion-microservicios y pulsar Enter.
Acción 2: Escribir fc /L 1.3-principios-diseno\docs\ADR-003-principios-diseno.md 1.4-organizacion-solucion\docs\ADR-003-principios-diseno.md y pulsar Enter.
Acción 3: Verificar que la salida muestra FC: no differences encountered.
Acción 4: Escribir fc /L 1.3-principios-diseno\docs\principios-diseno.md 1.4-organizacion-solucion\docs\principios-diseno.md y pulsar Enter.
Acción 5: Verificar que la salida muestra FC: no differences encountered.

Por qué se hace: la comparación con fc verifica que los documentos del punto 1.3 no se han modificado.

Errores comunes:

· Usar comp en lugar de fc: comp compara binarios y no muestra diferencias de texto.
· Olvidar el flag /L: fc compara en modo ASCII y muestra líneas largas sin ajuste.
· Comparar archivos de carpetas equivocadas: la comparación no es válida.

Verificación: los dos documentos del punto 1.3 no muestran diferencias.

Paso 22 — Verificar la estructura del punto 1.4

Acción 1: Escribir tree /F 1.4-organizacion-solucion y pulsar Enter.
Acción 2: Verificar que la salida muestra AceriaApp.sln, README.md, src\AceriaApp.Monolito.Api\Organizacion con las tres clases nuevas, tests\AceriaApp.Organizacion.Tests con los dos archivos de tests y docs con el ADR-001, ADR-002, ADR-003, ADR-004, el análisis comparativo, el documento de principios y el documento de organización.
Acción 3: Verificar que las carpetas Dominio, Infraestructura, Metricas y Diseno del punto 1.3 permanecen intactas.
Acción 4: Verificar que no existe ninguna carpeta labs ni ningún archivo metaprompt.md ni verificacion.md.

Por qué se hace: la verificación estructural confirma que el punto 1.4 añade archivos nuevos sin modificar los existentes.

Errores comunes:

· Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
· Modificar las carpetas del punto 1.3: viola la regla de no modificar.
· Omitir la carpeta Organizacion: los tests no encuentran las clases.

Verificación: la estructura contiene los archivos nuevos y conserva los del punto 1.3.

Paso 23 — Verificar el .gitignore del punto 1.4

Acción 1: Verificar que el archivo .gitignore existe en la carpeta del punto 1.4 (copiado del punto 1.3).
Acción 2: Abrir el archivo .gitignore con Visual Studio 2026.
Acción 3: Verificar que contiene las líneas bin/, obj/, *.user, *.suo, .vs/ y TestResults/.
Acción 4: Verificar que no se ha modificado respecto al punto 1.3.
Acción 5: Pulsar Ctrl+S si no hay cambios y cerrar el archivo.

Por qué se hace: el .gitignore se hereda del punto 1.3 sin modificaciones. La verificación confirma que no se ha tocado.

Errores comunes:

· Modificar el .gitignore copiado del punto 1.3: viola la regla de no modificar archivos del punto anterior.
· Eliminar alguna línea: el repositorio se llena de artefactos.
· Crear el .gitignore con otro nombre: el archivo no se reconoce.

Verificación: el .gitignore existe y contiene las seis líneas sin modificaciones.

Paso 24 — Hacer el commit del punto 1.4 y verificar el estado final

Acción 1: Escribir cd C:\cursos\AceriaApp y pulsar Enter.
Acción 2: Escribir git add . y pulsar Enter.
Acción 3: Escribir git commit -m "Punto 1.4: organización de la solución y convenciones de nombres" y pulsar Enter.
Acción 4: Escribir git log --oneline y pulsar Enter para verificar los cuatro commits.
Acción 5: Escribir git status y pulsar Enter para verificar que el árbol está limpio.

Por qué se hace: el commit del punto 1.4 registra el estado acumulado del proyecto. Los cuatro commits consecutivos permiten comparar los puntos 1.1, 1.2, 1.3 y 1.4 con git diff.

Errores comunes:

· No tener configurado user.name y user.email: git commit falla.
· Añadir la carpeta bin/ y obj/ al commit: el .gitignore lo evita.
· Hacer commit antes de git add .: no hay cambios en el índice.

Verificación: git log --oneline muestra cuatro commits: Punto 1.1, Punto 1.2, Punto 1.3 y Punto 1.4. git status muestra nothing to commit, working tree clean.

Errores comunes del ejercicio completo

Error Causa Solución
xcopy no copia subcarpetas Falta el flag /E Ejecutar xcopy origen destino /E /I /H /Y
IReadOnlyList<string> no se resuelve Falta el using System.Collections.Generic; Añadir el using al principio del archivo
Contains no se resuelve Falta el using System.Linq; Añadir el using al principio del archivo
Where no se resuelve Falta el using System.Linq; Añadir el using al principio del archivo
NullReferenceException en CatalogoProyectos No se inicializa _nombres Inicializar en el constructor
El validador no detecta nombres sin prefijo Falta la comprobación con StartsWith Revisar la primera comprobación de Validar

El validador acepta nombres con cuatro partes Falta la comprobación de longitud Revisar la comprobación partes.Length != 3
Should() no se resuelve Falta el paquete FluentAssertions Ejecutar dotnet add package FluentAssertions
El proyecto de tests no compila Falta la referencia al proyecto monolítico Ejecutar dotnet add reference
La compilación falla por el .dll bloqueado La API está en ejecución Detener la API con Ctrl+C

Reto resuelto paso a paso

Reto: añadir un método Resumen a CatalogoProyectos que devuelva el número de proyectos válidos, el número de inválidos y el porcentaje de validez, y un test que verifique el cálculo cuando hay tres válidos y dos inválidos.

Paso 1: En CatalogoProyectos.cs, añadir el siguiente record al final del archivo:

```
public record ResumenProyectos(int Validos, int Invalidos, double PorcentajeValidez);
```

Paso 2: Explicar línea por línea:

Línea 1: public record ResumenProyectos(→ declara un contrato inmutable.
Línea 2: int Validos, → número de proyectos válidos.
Línea 3: int Invalidos, → número de proyectos inválidos.
Línea 4: double PorcentajeValidez); → porcentaje de validez.

Paso 3: En CatalogoProyectos.cs, añadir el siguiente método dentro de la clase:

```
public ResumenProyectos Resumen(ValidadorConvencionNombres validador)
{
    var validos = _nombres.Count(n => validador.Validar(n).EsValido);
    var invalidos = _nombres.Count - validos;
    var total = _nombres.Count;
    var porcentaje = total == 0 ? 0 : (double)validos / total * 100;
    return new ResumenProyectos(validos, invalidos, porcentaje);
}
```

Paso 4: Explicar línea por línea:

Línea 1: public ResumenProyectos Resumen(ValidadorConvencionNombres validador) → declara el método.
Línea 2: { → abre el cuerpo.
Línea 3: var validos = _nombres.Count(n => validador.Validar(n).EsValido); → cuenta los válidos.
Línea 4: var invalidos = _nombres.Count - validos; → calcula los inválidos por diferencia.
Línea 5: var total = _nombres.Count; → obtiene el total.
Línea 6: var porcentaje = total == 0 ? 0 : (double)validos / total * 100; → calcula el porcentaje con protección contra división por cero.
Línea 7: return new ResumenProyectos(validos, invalidos, porcentaje); → devuelve el resumen.
Línea 8: } → cierra el cuerpo.

Paso 5: Guardar el archivo con Ctrl+S.

Paso 6: En CatalogoProyectosTests.cs, añadir el siguiente test:

```
[Fact]
public void Resumen_ConTresValidosYDosInvalidos_CalculaPorcentaje()
{
    var nombres = new List<string>
    {
     "AceriaApp.Ordenes.Api",
     "AceriaApp.Ordenes.Domain",
     "AceriaApp.Inventario.Api",
     "AceriaApp.Ordenes",
     "AceriaApp.Inventario.Controlador"
```

```
    };
    var catalogo = new CatalogoProyectos(nombres);
    var validador = new ValidadorConvencionNombres(
     "AceriaApp",
     new[] { "Api", "Domain", "Infrastructure", "Application", "Tests" });

    var resumen = catalogo.Resumen(validador);

   resumen.Validos.Should().Be(3);
   resumen.Invalidos.Should().Be(2);
   resumen.PorcentajeValidez.Should().Be(60.0);
}
```

Paso 7: Explicar línea por línea:

Línea 1: [Fact] → marca el método como test de xUnit.
Línea 2: public void Resumen_ConTresValidosYDosInvalidos_CalculaPorcentaje() → declara el test.
Línea 3: { → abre el cuerpo.
Línea 4: var nombres = new List<string> → crea la lista.
Línea 5: { → abre el inicializador.
Línea 6: "AceriaApp.Ordenes.Api", → primer nombre válido.
Línea 7: "AceriaApp.Ordenes.Domain", → segundo nombre válido.
Línea 8: "AceriaApp.Inventario.Api", → tercer nombre válido.
Línea 9: "AceriaApp.Ordenes", → primer nombre inválido por falta de tipo.
Línea 10: "AceriaApp.Inventario.Controlador" → segundo nombre inválido por tipo no permitido.
Línea 11: }; → cierra el inicializador.
Línea 12: var catalogo = new CatalogoProyectos(nombres); → instancia el catálogo.
Línea 13: var validador = new ValidadorConvencionNombres(...); → instancia el validador.
Línea 14: var resumen = catalogo.Resumen(validador); → invoca el método.
Línea 15: resumen.Validos.Should().Be(3); → verifica los válidos.
Línea 16: resumen.Invalidos.Should().Be(2); → verifica los inválidos.
Línea 17: resumen.PorcentajeValidez.Should().Be(60.0); → verifica el porcentaje.
Línea 18: } → cierra el cuerpo.

Paso 8: Ejecutar dotnet test tests\AceriaApp.Organizacion.Tests\AceriaApp.Organizacion.Tests.csproj y verificar que pasan los seis tests.

Paso 9: Verificar que el método Resumen no modifica ningún archivo del punto 1.3.

Paso 10: Hacer commit con git commit -m "Reto 1.4: método Resumen en CatalogoProyectos con cálculo de porcentaje".

Verificación: la salida de dotnet test muestra Passed: 6.

Resultado esperado

Al finalizar este punto, el alumno debe tener:

- La carpeta modulo-1-introduccion-microservicios/1.4-organizacion-solucion/ con la solución AceriaApp.sln que contiene cinco proyectos.
- El documento docs/organizacion-solucion.md con las tres decisiones y las convenciones.
- El documento docs/ADR-004-organizacion-solucion.md con la decisión sobre organización.
- Las clases ConvencionNombres, ValidadorConvencionNombres y CatalogoProyectos en src/AceriaApp.Monolito.Api/Organizacion/.
- El proyecto tests/AceriaApp.Organizacion.Tests con cinco tests (seis tras el reto).
- Los tests de los puntos 1.1, 1.2, 1.3 y 1.4 pasando conjuntamente con dotnet test AceriaApp.sln.
- Los ADR-001, ADR-002 y ADR-003 del punto 1.3 sin modificar.
- Cuatro commits en Git: del punto 1.1 al 1.4.

Conclusión y enlace al siguiente punto

El punto 1.4 ha definido la organización de la solución con un monorepo, una separación clara entre src/, tests/ y docs/, y convenciones de nombres de proyectos, espacios de nombres y commits. El proyecto cuenta ahora con un validador automático que audita las convenciones y prepara la extracción de microservicios. Esta base permite avanzar hacia el punto 1.5, donde se estudiarán las buenas prácticas de desarrollo de microservicios: idempotencia, versionado, contratos, documentación y gestión de errores.

Fin del punto 1.4.

Espero confirmación del usuario antes de continuar con el punto 1.5.

PARTE PRÁCTICA (27 minutos)

Ejercicio: Añadir al punto 1.4 un documento de buenas prácticas en docs/, una clase de validación de buenas prácticas en el proyecto monolítico, un test unitario que verifique la validación y el resumen del estado del proyecto al final del módulo 1. El punto 1.5 parte del contenido del punto 1.4 sin modificar ningún archivo existente: solo añade archivos nuevos.

Ubicación de trabajo: C:\cursos\AceriaApp\modulo-1-introduccion-microservicios\

Paso 1 — Crear la carpeta del punto 1.5 a partir del punto 1.4

Acción 1: Abrir la terminal integrada de Visual Studio 2026 con Ctrl+ñ (o Ctrl+`` en teclados no españoles).
Acción 2: Escribir cd C:\cursos\AceriaApp\modulo-1-introduccion-microservicios y pulsar Enter.
Acción 3: Escribir mkdir 1.5-buenas-practicas y pulsar Enter.
Acción 4: Escribir xcopy 1.4-organizacion-solucion 1.5-buenas-practicas /E /I /H /Y y pulsar Enter.
Acción 5: Escribir dir 1.5-buenas-practicas y pulsar Enter para verificar que se ha copiado el contenido.

Por qué se hace: cada punto del curso parte del contenido del punto anterior. El punto 1.5 se construye sobre el punto 1.4 copiando su contenido y añadiendo archivos nuevos sin modificar los existentes.

Errores comunes:

· Usar copy en lugar de xcopy: copy no copia carpetas de forma recursiva.
· Olvidar el flag /E: no se copian las subcarpetas vacías.
· Olvidar el flag /I: xcopy pregunta si el destino es un directorio cuando no existe.
· Olvidar el flag /Y: xcopy pide confirmación para sobrescribir.

Verificación: al ejecutar dir 1.5-buenas-practicas, deben aparecer AceriaApp.sln, README.md, src, tests y docs.

Paso 2 — Abrir la solución copiada en Visual Studio 2026

Acción 1: Escribir cd 1.5-buenas-practicas y pulsar Enter.
Acción 2: Escribir start AceriaApp.sln y pulsar Enter.

Acción 3: Verificar que Visual Studio 2026 abre la solución y muestra los proyectos heredados del punto 1.4.
Acción 4: Verificar que la solución compila con Ctrl+Shift+B y que la salida muestra Build succeeded..

Por qué se hace: abrir la solución copiada permite trabajar en el punto 1.5 sobre la base del punto 1.4. La compilación verifica que la copia es correcta.

Errores comunes:

· Abrir la solución del punto 1.4 en lugar de la del punto 1.5: los cambios se aplican al punto equivocado.
· No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
· Tener la API del punto 1.4 en ejecución: el puerto queda ocupado y la compilación falla.

Verificación: Visual Studio 2026 muestra la solución AceriaApp con los cinco proyectos heredados y la compilación es correcta.

Paso 3 — Verificar que la carpeta docs contiene los documentos del punto 1.4

Acción 1: En el panel Solution Explorer, expandir la carpeta docs del punto 1.5.
Acción 2: Verificar que contiene ADR-001, ADR-002, ADR-003, ADR-004, analisis-ventajas-inconvenientes.md, principios-diseno.md y organizacion-solucion.md.
Acción 3: Verificar que ninguno de los siete documentos se ha modificado.
Acción 4: Verificar que la carpeta docs no contiene ningún archivo nuevo del punto 1.5 todavía.

Por qué se hace: el punto 1.5 parte del contenido del punto 1.4, que ya incluye la carpeta docs con siete documentos. No se modifica ningún documento existente.

Errores comunes:

· Modificar alguno de los ADR o documentos existentes: viola la regla de no modificar archivos del punto anterior.
· Eliminar algún documento existente: pierde la trazabilidad del punto 1.4.
· Crear la carpeta docs si ya existe: mkdir falla.

Verificación: la carpeta docs contiene los siete documentos del punto 1.4 sin modificaciones.

Paso 4 — Crear el documento de buenas prácticas en docs

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta docs.
Acción 2: Seleccionar Add → New Item → Text File y nombrar el archivo buenas-practicas.md.
Acción 3: Escribir exactamente el siguiente contenido:

Buenas prácticas — Punto 1.5

Categorías

1. Contrato.
2. Operación.
3. Comunicación.
4. Evolución.

Buenas prácticas de contrato

- Exponer DTOs específicos en lugar de entidades de dominio.
- Versionar el contrato público en la URL con el prefijo `/api/v1/`.

Buenas prácticas de operación

- Idempotencia mediante cabecera `Idempotency-Key`.
- Gestión explícita de errores con códigos HTTP y problem details.

Buenas prácticas de comunicación

- Timeouts en todas las llamadas externas.
- Reintentos con retroceso exponencial.
- Circuit breaker para evitar saturación.

Buenas prácticas de evolución

- Mantener la compatibilidad hacia atrás de los contratos.
- Verificar la compatibilidad con tests automáticos.
- Mantener documentación viva con OpenAPI.

Aplicación al proyecto del curso

```
| Práctica | Aplicación |
|----------|------------|
| DTOs separados | `OrdenDto` en lugar de `Orden` |
| Versionado en URL | `/api/v1/ordenes` |
| Idempotencia | Cabecera `Idempotency-Key` |
| Errores explícitos | Problem details |
| Timeouts y reintentos | Polly |
| Circuit breaker | Polly |
| Contratos compatibles | Tests de compatibilidad |
| Documentación viva | OpenAPI |
```

Cierre del módulo 1

El proyecto AceriaApp dispone de:

- Monolito modular de referencia con API Minimal.
- Persistencia con EF Core 8.0.0 y SQL Server Express.
- Documentación OpenAPI con Swagger.
- Métricas de acoplamiento.
- Validador de límites de servicio.
- Validador de convenciones de nombres.
- Validador de buenas prácticas.
- Tests unitarios acumulados en cinco proyectos de tests.

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que el archivo aparece en la carpeta docs sin subrayados rojos.

Por qué se hace: el documento de buenas prácticas es el artefacto de docs específico del punto 1.5. Incluye el cierre del módulo 1 con el resumen del estado del proyecto. No modifica los siete documentos existentes del punto 1.4.

Errores comunes:

· Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
· Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
· Guardar el archivo en la raíz del punto en lugar de docs: rompe la estructura del repositorio.

Verificación: el archivo docs\buenas-practicas.md existe con las siete secciones.

Paso 5 — Crear la carpeta BuenasPracticas en el proyecto monolítico

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.Monolito.Api.
Acción 2: Seleccionar Add → New Folder y nombrar la carpeta BuenasPracticas.

Acción 3: Verificar que la carpeta BuenasPracticas aparece dentro de AceriaApp.Monolito.Api sin afectar a las carpetas Dominio, Infraestructura, Metricas, Diseno y Organizacion.
Acción 4: Verificar que no se ha modificado ningún archivo existente del punto 1.4.

Por qué se hace: la carpeta BuenasPracticas albergará las clases de validación de buenas prácticas. Se crea como carpeta nueva sin tocar las carpetas existentes.

Errores comunes:

· Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
· Modificar las carpetas existentes: viola la regla de no modificar archivos del punto anterior.
· Crear la carpeta con otro nombre: rompe la convención del curso.

Verificación: la carpeta BuenasPracticas existe dentro de AceriaApp.Monolito.Api y las carpetas existentes permanecen intactas.

Paso 6 — Crear la clase Practica

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta BuenasPracticas.
Acción 2: Seleccionar Add → Class y nombrar la clase Practica.cs.
Acción 3: Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.BuenasPracticas;

public class Practica
{
    public string Codigo { get; init; } = string.Empty;
    public string Categoria { get; init; } = string.Empty;
    public string Descripcion { get; init; } = string.Empty;
    public bool EstaAplicada { get; init; }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: la clase Practica modela una buena práctica con su código, categoría, descripción y estado de aplicación. Es la unidad de información que maneja el validador.

Errores comunes:

· Olvidar el namespace: la clase no se resuelve desde otros archivos.
· No inicializar las propiedades de tipo string con string.Empty: el compilador emite advertencias de nulabilidad.
· Usar set en lugar de init: permite modificar la instancia después de crearla.

Verificación: la clase Practica compila sin errores y expone cuatro propiedades.

Paso 7 — Crear la clase ValidadorBuenasPracticas

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta BuenasPracticas.
Acción 2: Seleccionar Add → Class y nombrar la clase ValidadorBuenasPracticas.cs.
Acción 3: Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.BuenasPracticas;

public class ValidadorBuenasPracticas
{
    private readonly IReadOnlyList<string> _categoriasPermitidas;
```

```
    public ValidadorBuenasPracticas(IReadOnlyList<string> categoriasPermitidas)
    {
     _categoriasPermitidas = categoriasPermitidas;
    }

    public ResultadoPracticas Validar(IEnumerable<Practica> practicas)
    {
        var errores = new List<string>();
        var lista = practicas.ToList();

        if (lista.Count == 0)
        {
       errores.Add("No se ha declarado ninguna práctica.");
        }

     foreach (var practica in lista)
        {
            if (!_categoriasPermitidas.Contains(practica.Categoria))
            {
          errores.Add($"Categoría no permitida: '{practica.Categoria}'.");
            }

            if (!practica.EstaAplicada)
            {
          errores.Add($"Práctica no aplicada: '{practica.Codigo}'.");
            }
        }

        return new ResultadoPracticas(errores.Count == 0, errores);
    }
}

public record ResultadoPracticas(bool EsValido, IReadOnlyList<string> Errores);
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: la clase ValidadorBuenasPracticas comprueba que todas las prácticas declaradas pertenecen a categorías permitidas y están aplicadas. Es el mecanismo de auditoría de buenas prácticas.

Errores comunes:

· Olvidar el using System.Collections.Generic;: IReadOnlyList<string> y List<string> no se resuelven.
· Olvidar el using System.Linq;: ToList() y Contains no se resuelven.
· No comprobar el caso de lista vacía: el validador devuelve válido sin prácticas.

Verificación: la clase ValidadorBuenasPracticas compila sin errores y expone un constructor, un método y un record.

Paso 8 — Añadir los using necesarios en ValidadorBuenasPracticas

Acción 1: En el panel Solution Explorer, hacer doble clic en ValidadorBuenasPracticas.cs.
Acción 2: Añadir las líneas using System.Collections.Generic; y using System.Linq; al principio del archivo si no están presentes.
Acción 3: Verificar que no hay subrayados rojos en IReadOnlyList<string> ni en ToList ni en Contains.
Acción 4: Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: los tipos IReadOnlyList<string> y List<string> pertenecen a System.Collections.Generic. Los métodos ToList y Contains pertenecen a System.Linq. En proyectos con ImplicitUsings desactivado, hay que añadir los usings explícitos.

Errores comunes:

· Añadir solo System.Collections.Generic y olvidar System.Linq: ToList y Contains no se resuelven.
· Añadir los usings después del namespace: el compilador no los reconoce.
· Olvidar los usings y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: ValidadorBuenasPracticas.cs compila sin errores y no muestra subrayados rojos.

Paso 9 — Crear la clase CatalogoPracticas

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta BuenasPracticas.
Acción 2: Seleccionar Add → Class y nombrar la clase CatalogoPracticas.cs.
Acción 3: Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.BuenasPracticas;

public class CatalogoPracticas
{
    private readonly List<Practica> _practicas;

    public CatalogoPracticas(List<Practica> practicas)
    {
     _practicas = practicas;
    }

    public IEnumerable<Practica> PracticasAplicadas()
    {
        return _practicas.Where(p => p.EstaAplicada);
    }

    public IEnumerable<Practica> PracticasPendientes()
    {
        return _practicas.Where(p => !p.EstaAplicada);
    }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: la clase CatalogoPracticas agrupa las prácticas del proyecto y permite filtrarlas por estado de aplicación.

Errores comunes:

· No inicializar _practicas en el constructor: la clase lanza NullReferenceException.
· Olvidar using System.Linq;: Where no se resuelve.
· Devolver List<Practica> en lugar de IEnumerable<Practica>: expone la implementación interna.

Verificación: la clase CatalogoPracticas compila sin errores y expone un constructor y dos métodos.

Paso 10 — Añadir el using necesario en CatalogoPracticas

Acción 1: En el panel Solution Explorer, hacer doble clic en CatalogoPracticas.cs.
Acción 2: Añadir la línea using System.Linq; al principio del archivo si no está presente.
Acción 3: Verificar que no hay subrayados rojos en Where.
Acción 4: Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: el método Where pertenece a System.Linq. En proyectos con ImplicitUsings desactivado, hay que añadir el using explícito.

Errores comunes:

· Añadir using System.Collections.Generic; en lugar de System.Linq: no resuelve Where.

· Añadir el using después del namespace: el compilador no lo reconoce.
· Olvidar el using y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: CatalogoPracticas.cs compila sin errores y no muestra subrayados rojos.

Paso 11 — Crear el proyecto de tests del punto 1.5

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta tests.
Acción 2: Seleccionar Add → New Project.
Acción 3: Seleccionar la plantilla xUnit Test Project, nombrar el proyecto AceriaApp.BuenasPracticas.Tests y pulsar Create.
Acción 4: Verificar que el proyecto aparece en la solución y que se ha creado la carpeta tests\AceriaApp.BuenasPracticas.Tests.

Por qué se hace: el punto 1.5 añade un proyecto de tests específico para las clases de buenas prácticas. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

Errores comunes:

· Crear el proyecto dentro de src en lugar de tests: rompe la estructura.
· Modificar los proyectos de tests del punto 1.4: viola la regla de no modificar archivos del punto anterior.
· Usar dotnet new mstest: el curso usa xUnit.

Verificación: el proyecto AceriaApp.BuenasPracticas.Tests aparece en la solución y su carpeta existe en tests\AceriaApp.BuenasPracticas.Tests.

Paso 12 — Añadir la referencia del proyecto de tests a la API monolítica

Acción 1: En la terminal, escribir dotnet add tests\AceriaApp.BuenasPracticas.Tests\AceriaApp.BuenasPracticas.Tests.csproj reference src\AceriaApp.Monolito.Api\AceriaApp.Monolito.Api.csproj y pulsar Enter.
Acción 2: Verificar que la salida muestra Reference added successfully.
Acción 3: Verificar que el archivo AceriaApp.BuenasPracticas.Tests.csproj contiene la referencia.

Por qué se hace: la referencia permite que el proyecto de tests acceda a las clases de BuenasPracticas del proyecto monolítico.

Errores comunes:

· Añadir la referencia en dirección inversa: crea una dependencia circular.
· Olvidar el nombre completo del archivo .csproj: el comando falla.
· Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

Verificación: el archivo AceriaApp.BuenasPracticas.Tests.csproj contiene un ProjectReference al proyecto monolítico.

Paso 13 — Añadir el paquete FluentAssertions al proyecto de tests

Acción 1: Escribir dotnet add tests\AceriaApp.BuenasPracticas.Tests\AceriaApp.BuenasPracticas.Tests.csproj package FluentAssertions y pulsar Enter.

Acción 2: Escribir dotnet list tests\AceriaApp.BuenasPracticas.Tests\AceriaApp.BuenasPracticas.Tests.csproj package y pulsar Enter.
Acción 3: Verificar que la salida muestra FluentAssertions con su versión.

Por qué se hace: FluentAssertions proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

Errores comunes:

· Instalar una versión preliminar: el curso usa la última estable.
· Olvidar el paquete: Should() no se resuelve en los tests.
· Añadir el paquete al proyecto equivocado: los tests no compilan.

Verificación: la salida de dotnet list package muestra FluentAssertions con una versión estable.

Paso 14 — Crear el archivo ValidadorBuenasPracticasTests.cs

Acción 1: En el panel Solution Explorer, expandir tests → AceriaApp.BuenasPracticas.Tests.
Acción 2: Hacer doble clic en UnitTest1.cs y renombrarlo a ValidadorBuenasPracticasTests.cs.
Acción 3: Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.BuenasPracticas;
using FluentAssertions;
using Xunit;

namespace AceriaApp.BuenasPracticas.Tests;

public class ValidadorBuenasPracticasTests
{
    private readonly ValidadorBuenasPracticas _validador = new(
     new[] { "Contrato", "Operación", "Comunicación", "Evolución" });

    [Fact]
    public void Validar_PracticasCompletas_DevuelveValido()
    {
        var practicas = new List<Practica>
        {
            new() { Codigo = "C-01", Categoria = "Contrato", EstaAplicada = true },
            new() { Codigo = "O-01", Categoria = "Operación", EstaAplicada = true },
            new() { Codigo = "M-01", Categoria = "Comunicación", EstaAplicada = true },
            new() { Codigo = "E-01", Categoria = "Evolución", EstaAplicada = true }
        };

        var resultado = _validador.Validar(practicas);

     resultado.EsValido.Should().BeTrue();
     resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_CategoriaNoPermitida_DevuelveError()
    {
        var practicas = new List<Practica>
        {
            new() { Codigo = "X-01", Categoria = "Inventada", EstaAplicada = true }
        };

        var resultado = _validador.Validar(practicas);

     resultado.EsValido.Should().BeFalse();
     resultado.Errores.Should().Contain(e => e.Contains("no permitida"));
    }

    [Fact]
    public void Validar_PracticaNoAplicada_DevuelveError()
    {
        var practicas = new List<Practica>
```

```
        {
            new() { Codigo = "C-01", Categoria = "Contrato", EstaAplicada = false }
        };

        var resultado = _validador.Validar(practicas);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no aplicada"));
    }

    [Fact]
    public void Validar_SinPracticas_DevuelveError()
    {
        var resultado = _validador.Validar(new List<Practica>());

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("ninguna práctica"));
    }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: los tests verifican que el validador acepta prácticas completas y rechaza prácticas con categoría no permitida, prácticas no aplicadas y listas vacías.

Errores comunes:

- Olvidar el using AceriaApp.Monolito.Api.BuenasPracticas;: las clases no se resuelven.
- Usar Assert.True en lugar de Should().BeTrue(): el curso usa FluentAssertions.
- No cubrir el caso de lista vacía: el test no verifica el mensaje de error.

Verificación: el archivo ValidadorBuenasPracticasTests.cs compila y contiene cuatro métodos de test.

Paso 15 — Crear el archivo CatalogoPracticasTests.cs

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.BuenasPracticas.Tests.
Acción 2: Seleccionar Add → Class y nombrar la clase CatalogoPracticasTests.cs.
Acción 3: Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.BuenasPracticas;
using FluentAssertions;
using Xunit;

namespace AceriaApp.BuenasPracticas.Tests;

public class CatalogoPracticasTests
{
    [Fact]
    public void PracticasAplicadas_FiltraCorrectamente()
    {
        var practicas = new List<Practica>
        {
            new() { Codigo = "C-01", Categoria = "Contrato", EstaAplicada = true },
            new() { Codigo = "O-01", Categoria = "Operación", EstaAplicada = false },
            new() { Codigo = "M-01", Categoria = "Comunicación", EstaAplicada = true }
        };
        var catalogo = new CatalogoPracticas(practicas);

        var aplicadas = catalogo.PracticasAplicadas().ToList();
        var pendientes = catalogo.PracticasPendientes().ToList();

        aplicadas.Should().HaveCount(2);
        pendientes.Should().HaveCount(1);
        pendientes[0].Codigo.Should().Be("O-01");
```

```
    }
}
```

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que la clase compila sin errores.

Por qué se hace: el test verifica que el catálogo filtra correctamente las prácticas aplicadas y las pendientes.

Errores comunes:

· Olvidar using System.Collections.Generic;: List<T> no se resuelve.
· Olvidar using System.Linq;: ToList() no se resuelve.
· No comprobar el código de la práctica pendiente: el test podría pasar por casualidad.

Verificación: el archivo CatalogoPracticasTests.cs compila y contiene un método de test.

Paso 16 — Compilar la solución completa

Acción 1: Escribir dotnet build AceriaApp.sln y pulsar Enter.
Acción 2: Verificar que la salida muestra Build succeeded. y 0 Warning(s), 0 Error(s).
Acción 3: Verificar que el tiempo de compilación es inferior a 180 segundos.

Por qué se hace: compilar la solución completa verifica que todos los proyectos, incluidos los nuevos, se resuelven correctamente.

Errores comunes:

· Tener la API en ejecución: el archivo .dll está bloqueado y la compilación falla.
· No restaurar paquetes: dotnet build restaura por defecto, pero si hay un error de red, hay que ejecutar dotnet restore primero.
· Usar una versión distinta del SDK: dotnet --version debe devolver 8.x.x.

Verificación: la salida debe mostrar Build succeeded. con 0 Error(s).

Paso 17 — Ejecutar todos los tests del punto 1.5

Acción 1: Escribir dotnet test tests\AceriaApp.BuenasPracticas.Tests\AceriaApp.BuenasPracticas.Tests.csproj y pulsar Enter.
Acción 2: Verificar que la salida muestra Passed! - Failed: 0, Passed: 5, Skipped: 0.
Acción 3: Verificar que el tiempo total es inferior a 30 segundos.

Por qué se hace: ejecutar los tests del punto 1.5 verifica que las clases de buenas prácticas funcionan correctamente.

Errores comunes:

· Ejecutar dotnet test sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
· No compilar antes: dotnet test compila por defecto, pero si hay errores, los tests no se ejecutan.
· Tener la API en ejecución: el puerto puede estar ocupado.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 5, Skipped: 0, Total: 5.

Paso 18 — Ejecutar todos los tests de la solución

Acción 1: Escribir dotnet test AceriaApp.sln --no-build y pulsar Enter.
Acción 2: Verificar que la salida muestra Passed! - Failed: 0, Passed: 23, Skipped: 0.
Acción 3: Verificar que el tiempo total es inferior a 120 segundos.

Por qué se hace: ejecutar todos los tests de la solución verifica que los tests de los puntos 1.1, 1.2, 1.3, 1.4 y 1.5 pasan conjuntamente.

Errores comunes:

· No haber compilado antes: --no-build falla si no hay artefactos.
· Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
· Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 23, Skipped: 0, Total: 23.

Paso 19 — Crear el README del punto 1.5

Acción 1: En la carpeta del punto 1.5-buenas-practicas, hacer clic derecho y seleccionar Add → New Item → Text File.
Acción 2: Nombrar el archivo README.md.
Acción 3: Escribir exactamente el siguiente contenido:

Punto 1.5 — Buenas prácticas

Objetivo
Comprender las buenas prácticas aplicables al desarrollo de microservicios y añadir al proyecto las clases de validación de buenas prácticas. Cerrar el módulo 1 con el resumen del estado del proyecto.

Estado del proyecto
- Solución `AceriaApp.sln` con seis proyectos: `AceriaApp.Monolito.Api`, `AceriaApp.Monolito.Tests`, `AceriaApp.Metricas.Tests`, `AceriaApp.Diseno.Tests`, `AceriaApp.Organizacion.Tests` y `AceriaApp.BuenasPracticas.Tests`.
- Documento `docs/buenas-practicas.md` con las cuatro categorías y el cierre del módulo 1.
- Clases `Practica`, `ValidadorBuenasPracticas` y `CatalogoPracticas` en `src/AceriaApp.Monolito.Api/BuenasPracticas/`.
- Cinco tests nuevos en `tests/AceriaApp.BuenasPracticas.Tests`.

Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Ejecutar `dotnet ef database update` en `src\AceriaApp.Monolito.Api`.
4. Ejecutar `dotnet run` en `src\AceriaApp.Monolito.Api`.
5. Navegar a `https://localhost:7xxx/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

Qué se ha añadido respecto al punto anterior
- Documento `docs/buenas-practicas.md`.
- Carpeta `BuenasPracticas` en el proyecto monolítico con tres clases nuevas.
- Proyecto `tests/AceriaApp.BuenasPracticas.Tests` con cinco tests nuevos.
- Cierre del módulo 1 con el resumen del estado del proyecto.
- No se ha modificado ningún archivo del punto 1.4.

Documentación
- [ADR-001](docs/ADR-001-monolito-vs-microservicios.md) (heredado del punto 1.1).
- [ADR-002](docs/ADR-002-metricas-acoplamiento.md) (heredado del punto 1.2).

- [ADR-003](docs/ADR-003-principios-diseno.md) (heredado del punto 1.3).
- [ADR-004](docs/ADR-004-organizacion-solucion.md) (heredado del punto 1.4).
- [Análisis de ventajas e inconvenientes](docs/analisis-ventajas-inconvenientes.md) (heredado del punto 1.2).
- [Principios de diseño](docs/principios-diseno.md) (heredado del punto 1.3).
- [Organización de la solución](docs/organizacion-solucion.md) (heredado del punto 1.4).
- [Buenas prácticas](docs/buenas-practicas.md).

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que el README del punto 1.4 no se ha modificado.

Por qué se hace: el README del punto 1.5 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 1.4.

Errores comunes:

· Modificar el README del punto 1.4: viola la regla de no modificar archivos del punto anterior.
· Omitir la sección Qué se ha añadido respecto al punto anterior: pierde la trazabilidad.
· Escribir el README en la carpeta del punto 1.4: rompe la separación.

Verificación: el archivo README.md existe en la carpeta del punto 1.5 con las cinco secciones.

Paso 20 — Crear el ADR-005 del punto 1.5

Acción 1: En el panel Solution Explorer, hacer clic derecho sobre la carpeta docs.
Acción 2: Seleccionar Add → New Item → Text File y nombrar el archivo ADR-005-buenas-practicas.md.
Acción 3: Escribir exactamente el siguiente contenido:

ADR-005: Adopción de buenas prácticas y cierre del módulo 1

Estado
Aceptado

Contexto
El proyecto AceriaApp ha completado los puntos 1.1 a 1.4 del módulo 1. Es necesario cerrar el módulo con la adopción explícita de buenas prácticas y con una auditoría automática de su aplicación.

Decisión
Se adoptan las buenas prácticas de contrato, operación, comunicación y evolución. Se implementa un validador automático que audita las prácticas declaradas. El módulo 1 se cierra con el resumen del estado del proyecto.

Alternativas consideradas
- Adoptar las buenas prácticas sin auditoría: descartado por falta de trazabilidad.
- Adoptar solo algunas categorías: descartado por incoherencia.

Consecuencias
- Positivas: auditoría automática, trazabilidad, preparación para el módulo 2.
- Negativas: el validador puede requerir ajuste de categorías.
- Mitigación: revisar las categorías al final de cada módulo.

Acción 4: Pulsar Ctrl+S para guardar el archivo.
Acción 5: Verificar que los ADR-001 a ADR-004 no se han modificado.

Por qué se hace: el ADR-005 documenta la decisión de adoptar buenas prácticas y de cerrar el módulo 1. Es un artefacto nuevo del punto 1.5 que no modifica los ADR existentes.

Errores comunes:

· Modificar los ADR anteriores: viola la regla de no modificar archivos del punto anterior.

· Omitir la sección Alternativas consideradas: el ADR pierde valor.
· Nombrar el archivo sin el prefijo ADR-005: rompe la convención.

Verificación: el archivo docs\ADR-005-buenas-practicas.md existe con las cinco secciones.

Paso 21 — Verificar que no se ha modificado ningún archivo del punto 1.4

Acción 1: En la terminal, escribir cd C:\cursos\AceriaApp\modulo-1-introduccion-microservicios y pulsar Enter.
Acción 2: Escribir fc /L 1.4-organizacion-solucion\docs\ADR-004-organizacion-solucion.md 1.5-buenas-practicas\docs\ADR-004-organizacion-solucion.md y pulsar Enter.
Acción 3: Verificar que la salida muestra FC: no differences encountered.
Acción 4: Escribir fc /L 1.4-organizacion-solucion\docs\organizacion-solucion.md 1.5-buenas-practicas\docs\organizacion-solucion.md y pulsar Enter.
Acción 5: Verificar que la salida muestra FC: no differences encountered.

Por qué se hace: la comparación con fc verifica que los documentos del punto 1.4 no se han modificado.

Errores comunes:

· Usar comp en lugar de fc: comp compara binarios y no muestra diferencias de texto.
· Olvidar el flag /L: fc compara en modo ASCII y muestra líneas largas sin ajuste.
· Comparar archivos de carpetas equivocadas: la comparación no es válida.

Verificación: los dos documentos del punto 1.4 no muestran diferencias.

Paso 22 — Verificar la estructura del punto 1.5

Acción 1: Escribir tree /F 1.5-buenas-practicas y pulsar Enter.
Acción 2: Verificar que la salida muestra AceriaApp.sln, README.md, src\AceriaApp.Monolito.Api\BuenasPracticas con las tres clases nuevas, tests\AceriaApp.BuenasPracticas.Tests con los dos archivos de tests y docs con el ADR-001, ADR-002, ADR-003, ADR-004, ADR-005, el análisis comparativo, el documento de principios, el documento de organización y el documento de buenas prácticas.
Acción 3: Verificar que las carpetas Dominio, Infraestructura, Metricas, Diseno y Organizacion del punto 1.4 permanecen intactas.
Acción 4: Verificar que no existe ninguna carpeta labs ni ningún archivo metaprompt.md ni verificacion.md.

Por qué se hace: la verificación estructural confirma que el punto 1.5 añade archivos nuevos sin modificar los existentes.

Errores comunes:

· Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
· Modificar las carpetas del punto 1.4: viola la regla de no modificar.
· Omitir la carpeta BuenasPracticas: los tests no encuentran las clases.

Verificación: la estructura contiene los archivos nuevos y conserva los del punto 1.4.

Paso 23 — Verificar el .gitignore del punto 1.5

Acción 1: Verificar que el archivo .gitignore existe en la carpeta del punto 1.5 (copiado del punto 1.4).
Acción 2: Abrir el archivo .gitignore con Visual Studio 2026.
Acción 3: Verificar que contiene las líneas bin/, obj/, *.user, *.suo, .vs/ y TestResults/.
Acción 4: Verificar que no se ha modificado respecto al punto 1.4.
Acción 5: Pulsar Ctrl+S si no hay cambios y cerrar el archivo.

Por qué se hace: el .gitignore se hereda del punto 1.4 sin modificaciones. La verificación confirma que no se ha tocado.

Errores comunes:

· Modificar el .gitignore copiado del punto 1.4: viola la regla de no modificar archivos del punto anterior.
· Eliminar alguna línea: el repositorio se llena de artefactos.
· Crear el .gitignore con otro nombre: el archivo no se reconoce.

Verificación: el .gitignore existe y contiene las seis líneas sin modificaciones.

Paso 24 — Hacer el commit del punto 1.5 y verificar el estado final del módulo 1

Acción 1: Escribir cd C:\cursos\AceriaApp y pulsar Enter.
Acción 2: Escribir git add . y pulsar Enter.
Acción 3: Escribir git commit -m "Punto 1.5: buenas prácticas y cierre del módulo 1" y pulsar Enter.
Acción 4: Escribir git log --oneline y pulsar Enter para verificar los cinco commits.
Acción 5: Escribir git status y pulsar Enter para verificar que el árbol está limpio.

Por qué se hace: el commit del punto 1.5 cierra el módulo 1. Los cinco commits consecutivos permiten comparar el estado del proyecto entre puntos consecutivos y entre módulos completos.

Errores comunes:

· No tener configurado user.name y user.email: git commit falla.
· Añadir la carpeta bin/ y obj/ al commit: el .gitignore lo evita.
· Hacer commit antes de git add .: no hay cambios en el índice.

Verificación: git log --oneline muestra cinco commits: Punto 1.1, Punto 1.2, Punto 1.3, Punto 1.4 y Punto 1.5. git status muestra nothing to commit, working tree clean.

Errores comunes del ejercicio completo

Error Causa Solución
xcopy no copia subcarpetas Falta el flag /E Ejecutar xcopy origen destino /E /I /H /Y
IReadOnlyList<string> no se resuelve Falta el using System.Collections.Generic; Añadir el using al principio del archivo
ToList no se resuelve Falta el using System.Linq; Añadir el using al principio del archivo
Contains no se resuelve Falta el using System.Linq; Añadir el using al principio del archivo
Where no se resuelve Falta el using System.Linq; Añadir el using al principio del archivo
NullReferenceException en CatalogoPracticas No se inicializa _practicas Inicializar en el constructor
El validador no detecta categorías inválidas Falta la comprobación con Contains Revisar la primera comprobación del bucle
El validador no detecta prácticas no aplicadas Falta la comprobación !practica.EstaAplicada Revisar la segunda comprobación del bucle

Should() no se resuelve Falta el paquete FluentAssertions Ejecutar dotnet add package FluentAssertions
El proyecto de tests no compila Falta la referencia al proyecto monolítico Ejecutar dotnet add reference

---

Reto resuelto paso a paso

Reto: añadir un método Resumen a CatalogoPracticas que devuelva el número de prácticas aplicadas, el número de pendientes y el porcentaje de aplicación, y un test que verifique el cálculo cuando hay tres aplicadas y dos pendientes.

Paso 1: En CatalogoPracticas.cs, añadir el siguiente record al final del archivo:

```
public record ResumenPracticas(int Aplicadas, int Pendientes, double PorcentajeAplicacion);
```

Paso 2: Explicar línea por línea:

Línea 1: public record ResumenPracticas(→ declara un contrato inmutable.
Línea 2: int Aplicadas, → número de prácticas aplicadas.
Línea 3: int Pendientes, → número de prácticas pendientes.
Línea 4: double PorcentajeAplicacion); → porcentaje de aplicación.

Paso 3: En CatalogoPracticas.cs, añadir el siguiente método dentro de la clase:

```
public ResumenPracticas Resumen()
{
    var aplicadas = _practicas.Count(p => p.EstaAplicada);
    var pendientes = _practicas.Count - aplicadas;
    var total = _practicas.Count;
    var porcentaje = total == 0 ? 0 : (double)aplicadas / total * 100;
    return new ResumenPracticas(aplicadas, pendientes, porcentaje);
}
```

Paso 4: Explicar línea por línea:

Línea 1: public ResumenPracticas Resumen() → declara el método.
Línea 2: { → abre el cuerpo.
Línea 3: var aplicadas = _practicas.Count(p => p.EstaAplicada); → cuenta las aplicadas.
Línea 4: var pendientes = _practicas.Count - aplicadas; → calcula las pendientes por diferencia.
Línea 5: var total = _practicas.Count; → obtiene el total.
Línea 6: var porcentaje = total == 0 ? 0 : (double)aplicadas / total * 100; → calcula el porcentaje con protección contra división por cero.
Línea 7: return new ResumenPracticas(aplicadas, pendientes, porcentaje); → devuelve el resumen.
Línea 8: } → cierra el cuerpo.

Paso 5: Guardar el archivo con Ctrl+S.

Paso 6: En CatalogoPracticasTests.cs, añadir el siguiente test:

```
[Fact]
public void Resumen_ConTresAplicadasYDosPendientes_CalculaPorcentaje()
{
    var practicas = new List<Practica>
    {
        new() { Codigo = "C-01", Categoria = "Contrato", EstaAplicada = true },
        new() { Codigo = "C-02", Categoria = "Contrato", EstaAplicada = true },
        new() { Codigo = "O-01", Categoria = "Operación", EstaAplicada = true },
        new() { Codigo = "M-01", Categoria = "Comunicación", EstaAplicada = false },
        new() { Codigo = "E-01", Categoria = "Evolución", EstaAplicada = false }
    };
    var catalogo = new CatalogoPracticas(practicas);

    var resumen = catalogo.Resumen();

   resumen.Aplicadas.Should().Be(3);
   resumen.Pendientes.Should().Be(2);
```

```
    resumen.PorcentajeAplicacion.Should().Be(60.0);
}
```

Paso 7: Explicar línea por línea:

Línea 1: [Fact] → marca el método como test de xUnit.
Línea 2: public void Resumen_ConTresAplicadasYDosPendientes_CalculaPorcentaje() → declara el test.
Línea 3: { → abre el cuerpo.
Línea 4: var practicas = new List<Practica> → crea la lista.
Línea 5: { → abre el inicializador.
Línea 6: new() { Codigo = "C-01", ..., EstaAplicada = true }, → primera práctica aplicada.
Línea 7: new() { Codigo = "C-02", ..., EstaAplicada = true }, → segunda práctica aplicada.
Línea 8: new() { Codigo = "O-01", ..., EstaAplicada = true }, → tercera práctica aplicada.
Línea 9: new() { Codigo = "M-01", ..., EstaAplicada = false }, → primera práctica pendiente.
Línea 10: new() { Codigo = "E-01", ..., EstaAplicada = false } → segunda práctica pendiente.
Línea 11: }; → cierra el inicializador.
Línea 12: var catalogo = new CatalogoPracticas(practicas); → instancia el catálogo.
Línea 13: var resumen = catalogo.Resumen(); → invoca el método.
Línea 14: resumen.Aplicadas.Should().Be(3); → verifica las aplicadas.
Línea 15: resumen.Pendientes.Should().Be(2); → verifica las pendientes.
Línea 16: resumen.PorcentajeAplicacion.Should().Be(60.0); → verifica el porcentaje.
Línea 17: } → cierra el cuerpo.

Paso 8: Ejecutar dotnet test tests\AceriaApp.BuenasPracticas.Tests\AceriaApp.BuenasPracticas.Tests.csproj y verificar que pasan los seis tests.

Paso 9: Verificar que el método Resumen no modifica ningún archivo del punto 1.4.

Paso 10: Hacer commit con git commit -m "Reto 1.5: método Resumen en CatalogoPracticas con cálculo de porcentaje".

Verificación: la salida de dotnet test muestra Passed: 6.

Resultado esperado

Al finalizar este punto, el alumno debe tener:

- La carpeta modulo-1-introduccion-microservicios/1.5-buenas-practicas/ con la solución AceriaApp.sln que contiene seis proyectos.
- El documento docs/buenas-practicas.md con las cuatro categorías y el cierre del módulo 1.
- El documento docs/ADR-005-buenas-practicas.md con la decisión sobre buenas prácticas.
- Las clases Practica, ValidadorBuenasPracticas y CatalogoPracticas en src/AceriaApp.Monolito.Api/BuenasPracticas/.
- El proyecto tests/AceriaApp.BuenasPracticas.Tests con cinco tests (seis tras el reto).
- Los tests de los puntos 1.1 a 1.5 pasando conjuntamente con dotnet test AceriaApp.sln.
- Los ADR-001 a ADR-004 del punto 1.4 sin modificar.
- Cinco commits en Git: del punto 1.1 al 1.5.
- El módulo 1 cerrado con el resumen del estado del proyecto en docs/buenas-practicas.md.

Conclusión y enlace al siguiente punto

El punto 1.5 ha cerrado el módulo 1 con la adopción explícita de buenas prácticas de contrato, operación, comunicación y evolución. El proyecto AceriaApp dispone ahora de un monolito modular con

API Minimal, persistencia con EF Core 8.0.0 y SQL Server Express, documentación OpenAPI, métricas de acoplamiento, validador de límites de servicio, validador de convenciones de nombres, validador de buenas prácticas y 23 tests unitarios acumulados. El módulo 1 queda cerrado con el resumen del estado del proyecto en docs/buenas-practicas.md. Esta base permite avanzar hacia el módulo 2, donde se estudiará el desarrollo de APIs REST con ASP.NET Core: diseño de APIs, recursos y verbos HTTP, validación de peticiones y documentación OpenAPI.

Fin del punto 1.5 y del módulo 1.

Espero confirmación del usuario antes de continuar con el módulo 2, punto 2.1 (Diseño de APIs REST).